diag180: Counting polyominoes with 180° rotational symmetry by a transfer matrix along diagonals

(production version)

Günter Rote Claude

November 14, 2026

Abstract

This program counts the fixed polyominoes with n cells that are mapped to themselves by a rotation by 180°, for all n up to a given bound N. It builds one half of the polyomino row by row along the *diagonals* of the square grid and keeps, for every intermediate boundary configuration, only the number of ways to reach it. Configurations that cannot be completed with at most N cells are discarded with the help of a lower bound on the number of missing cells. For small n we have checked that this bound is exact. The numbers are one of the ingredients that are needed to extend the count of *free* polyominoes (OEIS A000105) beyond $n = 59$. The program is written in CWEB [7].

Contents

1 Introduction **2**
1.1 Symmetry classes and cases . . . 3
1.2 Diagonal coordinates . . . 4
1.3 Wide and tall polyominoes . . . 5
1.4 The half polyomino . . . 5

2 The state **6**
2.1 Components: central, paired, and floating . . . 6
2.2 The unpacked state . . . 7
2.3 The working state . . . 8
2.4 Joining two components . . . 9
2.5 Removing a cell from the frontier . . . 10
2.6 A transition . . . 10

3 Packed keys **11**
3.1 The length of a key . . . 11
3.2 One bit per component is enough . . . 12
3.3 Encoding and decoding . . . 12
3.4 Positions that can hold a frontier cell . . . 14
3.5 Compaction into a bit string . . . 14

4 Counters **15**

5 The tables **16**
5.1 The hash function . . . 17
5.2 Growing, initializing, and finishing a table . . . 18
5.3 Finding and creating states . . . 19
5.4 Count records . . . 20

6 The lower bound on the number of cells necessary to reach the boundary **21**
6.1 Connections on the diagonal lattice . . . 21
6.2 The cactus of gaps . . . 22
6.3 Reaching the sides and the top . . . 22
6.4 Functions for the height terms . . . 23
6.5 Examples . . . 25

6.6 Putting the bound together . . . 26
6.7 The complete bound . . . 30

7 The main loop **30**
7.1 Reporting . . . 31
7.2 Treating one case . . . 32
7.3 Initial states . . . 34
7.4 Processing a row . . . 36
7.5 Lookahead . . . 37
7.6 Reflection . . . 38

8 The main program **39**
8.1 Limits . . . 40
8.2 Running the cases and writing the results . . . 41

9 Results and experiments **43**
9.1 Correctness . . . 43
9.2 Tightness of the bound . . . 43
9.3 Growth . . . 43
9.4 Values . . . 44
9.5 Odd and even numbers of cells . . . 44
9.6 Saving space in larger runs . . . 44

10 References **46**

1 Introduction

A *polyomino* is a finite set of cells of the square grid that is connected through common edges. *Fixed* polyominoes are counted up to translation only; *free* polyominoes are counted up to translation, rotation and reflection. By Burnside's lemma, the number of free polyominoes with n cells is

$$\frac{1}{8}\sum_{g\in D_4} \mathrm{fix}_g(n),$$

where $\mathrm{fix}_g(n)$ is the number of fixed polyominoes with n cells that are invariant under the symmetry g of the square, up to translation. For the identity this is the number $A(n)$ of all fixed polyominoes (OEIS A001168 [8]), which is known up to $n = 70$ by the work of Barequet and Ben-Shachar [1, 2]. (The value for $n = 67$ printed in [1] is too large by 2; it is corrected in [2].) The other seven terms count symmetric polyominoes. The number of free polyominoes is currently known up to $n = 59$ (Shirakawa [10]), and in that computation the polyominoes with a 180° rotational symmetry were the bottleneck: they were enumerated one by one.

This program computes $\mathrm{fix}_\rho(n)$ for the rotation ρ by 180° with a transfer-matrix method, in the spirit of Jensen's algorithm [4, 5] and of Knuth's programs POLYNUM and POLYSLAVE [6], but with three special features:

1. We do not build the polyomino P but its "half", the quotient of P under ρ. The connectivity information in the state has to keep track of how each component is related to its own mirror image.

2. The half is built along the diagonals $x + y =$ const of the grid, as in the work of Barequet and Ben-Shachar on fixed polyominoes [1]. Connections between neighboring components then use *disjoint sets of columns*, which makes it possible to compute the cost of the cheapest completion of a state almost exactly.

3. States that cannot lead to a polyomino with at most N cells are pruned, using this cost (Section 6). In our experiments the growth of the number of states is about 1.25 per cell, compared to roughly 1.43 for counting all fixed polyominoes with the same technique.

Polyominoes with symmetries are easier to count, because they are more restricted and hence fewer in number. We can expect the number of polyominoes that have a rotational symmetry to be roughly $\sqrt{A(n)}$, because the inclusion or exclusion of one cell decides about its mirror cell as well (except possibly for one cell at the center of the rotation), and thus the number of independent choices is only half as big as without

any constraints. Our results (Table 3) confirm this very well: the quotient $\mathrm{fix}_\rho(n)/\sqrt{A(n)}$ lies between 1.60 and 1.64 for every even n with $8 \le n \le 70$, and between 0.78 and 0.87 for every odd n with $9 \le n \le 69$. In both cases it decreases slowly; some values are shown in Table 1. Generally, we can expect the number of polyominoes that have the symmetries in some given symmetry group of order g to be roughly $\sqrt[g]{A(n)}$.

n	$\mathrm{fix}_\rho(n)$	$\sqrt{A(n)}$	quotient	n	$\mathrm{fix}_\rho(n)$	$\sqrt{A(n)}$	quotient
10	310	190	1.624	11	316	367	0.859
20	247196	151541	1.631	21	249283	298301	0.836
30	$2.240 \cdot 10^8$	$1.376 \cdot 10^8$	1.627	31	$2.239 \cdot 10^8$	$2.730 \cdot 10^8$	0.820
40	$2.144 \cdot 10^{11}$	$1.323 \cdot 10^{11}$	1.621	41	$2.130 \cdot 10^{11}$	$2.634 \cdot 10^{11}$	0.809
50	$2.117 \cdot 10^{14}$	$1.312 \cdot 10^{14}$	1.614	51	$2.094 \cdot 10^{14}$	$2.618 \cdot 10^{14}$	0.800
60	$2.132 \cdot 10^{17}$	$1.326 \cdot 10^{17}$	1.608	61	$2.102 \cdot 10^{17}$	$2.652 \cdot 10^{17}$	0.793
70	$2.178 \cdot 10^{20}$	$1.360 \cdot 10^{20}$	1.602	69	$5.353 \cdot 10^{19}$	$6.797 \cdot 10^{19}$	0.788

Table 1: The number of polyominoes with a 180° rotational symmetry compared with the square root of the number $A(n)$ of all fixed polyominoes, for even n (left) and odd n (right).

The program is called in one of the following ways:

```
diag180 [options] N           all cases; prints n and fix(n) for n = 1..N
diag180 [options] N Mx My umax   one case (see below); prints n, #portrait,
#square
```

The options are described in Section 8.

¶ The overall structure of the program is conventional.

2 ⟨ Include files 3 ⟩
⟨ Type definitions 13 ⟩
⟨ Global variables 8 ⟩
⟨ Subroutines 9 ⟩
⟨ The main program 101 ⟩

3 ¶ ⟨ Include files 3 ⟩ ≡

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>
```

This code is used in chunk 2.

¶ The macros *max* and *min* are used in many places. Their arguments are evaluated twice, so they must not have side effects.

4 **#define** $max(a, b)$ $((a) > (b)\ ?\ (a) : (b))$
#define $min(a, b)$ $((a) < (b)\ ?\ (a) : (b))$

1.1 Symmetry classes and cases

The rotation ρ of a polyomino P that is invariant under it has a center, and after a translation we may assume that

$$\rho(x, y) = (M_x - x,\ M_y - y) \qquad \text{with } M_x, M_y \in \{0, 1\}.$$

The point (M_x, M_y) is the mirror image of the origin, and the center of the rotation is the point $(M_x/2, M_y/2)$. There are three types of centers:

- $(M_x, M_y) = (0, 0)$: the center of a cell. The center cell is the only cell that is its own image. Hence n is odd if the center cell belongs to P, and even if it does not (as for a ring around an empty center). (Type R180C in Shirakawa's notation [10].)

- $(M_x, M_y) = (1,0)$ or $(0,1)$: the midpoint of a vertical or horizontal edge. Here n is even. The two cases are mirror images of each other under $x \leftrightarrow y$ and give the same counts; we compute only $(1,0)$ and count it twice. (Type R180M.)
- $(M_x, M_y) = (1,1)$: a vertex of the grid; n is even. (Type R180V.)

In the OEIS [8], the three counts are the sequences A390653 (type C), A390654 (type M, for $2n$ cells and for horizontal edges only, which is half of our count) and A390655 (type V, for $2n$ cells), with terms up to 59 and 58 cells from Shirakawa's computation.

These sets are disjoint, and $\mathrm{fix}_\rho(n)$ is the sum of the counts over all centers. For odd n, only the center type C contributes, and in our experiments the run for $N = 2k+1$ needed no larger tables than the run for $N = 2k$.

1.2 Diagonal coordinates

We use the coordinates

$$u = x - y, \qquad v = x + y,$$

so that u increases to the right and v increases upward when the grid is turned by 45°; the cells appear as diamonds in the pictures. A cell (u, v) exists whenever $u+v$ is even, and its four neighbors are $(u \pm 1, v \pm 1)$. In our drawings, the point with coordinates (u, v) lies at the center of its cell. The set of cells with a fixed v is a *row* (a diagonal of the original grid); the set of cells with fixed u is a *column*. A neighbor of a cell is always in an adjacent row *and* in an adjacent column.

In these coordinates,

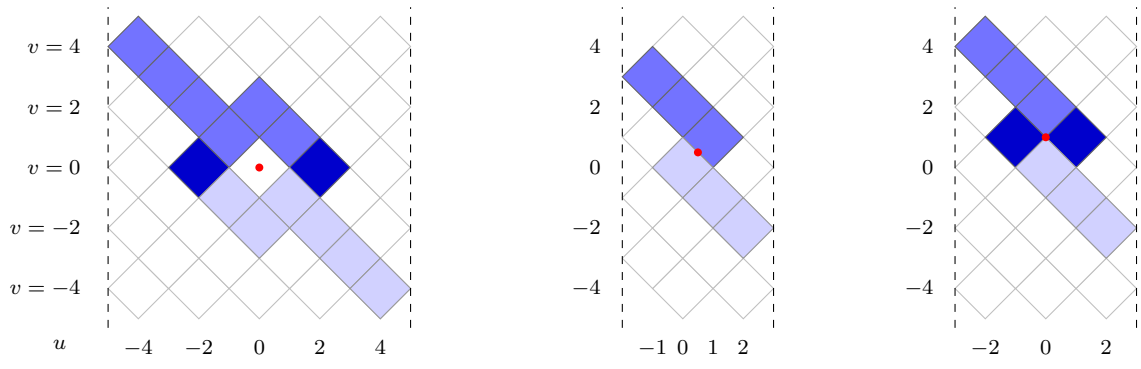
$$\rho(u, v) = (M_u - u,\ M_v - v) \qquad \text{with } M_u = M_x - M_y \text{ and } M_v = M_x + M_y,$$

and the center is the point $(M_u/2, M_v/2)$. The pairs that correspond to the three cases $(M_x, M_y) = (0,0)$, $(1,0)$, $(1,1)$ are $(M_u, M_v) = (0,0)$, $(1,1)$, $(0,2)$, see Figure 1.

Let $u_{\max}$ be the largest u-coordinate of P. By symmetry the smallest is $u_{\min} = M_u - u_{\max}$. A *case* is a triple $(M_x, M_y, u_{\max})$: we count the symmetric polyominoes that live in the strip $u_{\min} \le u \le u_{\max}$ of

$$W = u_{\max} - u_{\min} + 1 = 2u_{\max} - M_u + 1$$

columns and touch its two sides. (By symmetry, touching one side implies touching the other.) Inside the program the columns are numbered by their *position* $i = u - u_{\min}$, $0 \le i < W$, and the image of position i is position $W - 1 - i$.



(a) $(M_x, M_y) = (0,0)$, $(M_u, M_v) = (0,0)$ (b) $(1,0)$, $(1,1)$ (c) $(1,1)$, $(0,2)$

Figure 1: Symmetric polyominoes in diagonal coordinates for the three types of centers (red dot). The cells of the half P^+ (Section 1.4) are drawn in blue, their images in light blue, and the cells of the middle row are dark. (a) Center in a cell, $(M_x, M_y) = (0,0)$: a polyomino with 14 cells, a 3×3 ring around the empty center with two arms. The strip has $W = 9$ columns, $u = -4 \,..\, 4$, and the middle row is $v = 0$. (b) Center on the midpoint of a vertical edge, $(M_x, M_y) = (1,0)$: 6 cells, $W = 4$, $u = -1 \,..\, 2$. There is no middle row; there is a twin row $v = 1$, which is adjacent to its image, the row $v = 0$. (c) Center at a vertex, $(M_x, M_y) = (1,1)$: 8 cells, a 2×2 block with two arms, $W = 5$, $u = -2 \,..\, 2$, middle row $v = 1$.

1.3 Wide and tall polyominoes

The reflection $y \to -y$ exchanges v and u. It maps polyominoes with a 180° symmetry to polyominoes with a 180° symmetry, and it exchanges the extent in the v-direction (the number of rows, *vspan*) with the extent in the u-direction ($uspan = W$). Following Jensen [4] and his followers, we therefore generate only the polyominoes with $uspan \le vspan$, and we count those with $uspan < vspan$ twice:

$$\mathrm{fix}_\rho(n) = \sum_{\text{cases}} \big(2 \cdot \#\{uspan < vspan\} + \#\{uspan = vspan\}\big).$$

This has two advantages.

First, the strips are narrow. Going from a cell to a neighbor changes both u and v by 1, and hence $uspan + vspan \le 2n$ for a polyomino with n cells. With $uspan \le vspan$ this gives $W \le n$, so that we have to treat the strips with $W \le N$ columns. However, the wide strips among them are cheap: a polyomino that is W columns wide and at least W rows high needs many cells just for reaching the two sides, the top, and the bottom, so that few cells remain for variation, and the pruning leaves few states. The narrow strips are cheap as well, because the number of states is limited by the width. The work is concentrated in the strips of intermediate width, around $W \approx 0.4N$. In the run for $N = 71$, the largest tables occurred for $W = 28$ and $W = 29$ (15 to 20 million states), the strips with $22 \le W \le 34$ took 83 % of the running time, and the strips with $W > 40$ less than 3 %. (The three types of centers C, M, and V took 37 %, 42 %, and 21 % of the time.)

Second, the requirement that P must still grow to a height of at least W rows strengthens the pruning (Section 6.3).

¶ The geometry of the current case is kept in global variables.

8
```
#define MAXW 160    /* maximum number of columns of the strip */
⟨ Global variables 8 ⟩ ≡
  static int NMAX;    /* the bound N on the number of cells */
  static int M_x, M_y;    /* the center of the rotation is (M_x/2, M_y/2) */
  static int M_u, M_v;    /* the same in diagonal coordinates: M_u = M_x − M_y, M_v = M_x + M_y */
  static int u_min, u_max;    /* the strip: u_min ≤ u ≤ u_max */
  static int W;    /* number of columns of the strip */
  static int TWIN;    /* M_v mod 2 */
  static int NMAX_plus_1;    /* length of a vector of counters, NMAX + 1 */
```
See also chunks 14, 23, 34, 40, 58*, 74, 78, 97, and 100*

This code is used in chunk 2.

1.4 The half polyomino

Let the *upper half* P^+ be the set of cells of P with $v > M_v/2$, together with the cells of the *middle row* $v = M_v/2$ if M_v is even. Then $P = P^+ \cup \rho(P^+)$. The program constructs P^+ row by row, with increasing v, starting at the middle, and within each row from left to right, deciding for each cell whether it belongs to P^+ or not.

Every cell of P^+ outside the middle row stands for two cells of P and contributes 2 to the size n. We call the contribution to n the *weight*. If M_v is even, the middle row is mapped to itself. We do not build it cell by cell; we start with all possible symmetric subsets of the middle row. If M_v is odd, there is no middle row, but the first row $v_0 = (M_v + 1)/2$ of P^+ is adjacent to its own image, the row $v_0 - 1$. We call it the *twin row*. It is not built cell by cell either: we start with all possible subsets of the twin row.

After k rows beyond the middle have been built, the polyomino extends over $2k + 1$ rows if M_v is even and over $2k$ rows if M_v is odd.

9
```
⟨ Subroutines 9 ⟩ ≡
  static int vspan(int k)
  {
    return 2 * k + 1 − TWIN;
  }
```
See also chunks 15, 16, 17, 18, 20, 21, 26, 28, 29, 30, 31, 32, 35, 36, 38, 39, 41, 44, 45, 46, 47, 48, 49, 59, 60, 61, 62, 63, 64*, 65, 67, 73*, 77, 79, and 80

This code is used in chunk 2.

2 The state

At any time during the construction, some cells have been decided and the others are still open. The boundary between them is a zigzag line: if the cells of row v have been decided up to position i, the most recently decided cell in each column is in row v for the positions $\leq i$ and in row $v-1$ for the positions $> i$ (always only for the columns of the right parity). These cells form the *frontier*. The place where the frontier steps down from row v to row $v-1$ is called the *kink*. A frontier cell is dropped from the frontier as soon as both of its upper neighbors have been decided, since nothing can be attached to it any more. (See Figure 2.)

It is useful to imagine that we are simultanously building the polyomino downward from the center. Thus, there is a lower frontier, at the mirror position of the upper frontier, and if we decide to place a cell at the kink, its mirror will be placed below the lower frontier.

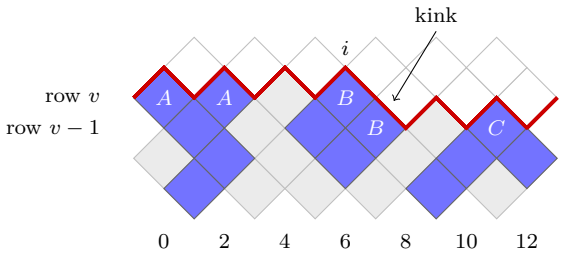


Figure 2: The situation after position $i = 6$ of row v has been decided. The kink is at position 7. Blue cells belong to P^+, gray cells are decided and empty, white cells are undecided. The thick red curve separates the decided cells from the unprocessed cells, and the frontier consists of all cells that touch an edge of the red curve from below: the cells of row v at the positions $0, 2, 4, 6$ and the cells of row $v-1$ at the positions $7, 9, 11$. Five of them are occupied; they form three components A, B, C with four runs. The frontier steps down at the kink, right of position i.

2.1 Components: central, paired, and floating

Consider the part of P that is known so far: the decided cells of P^+ and their images. From now on, the word *component* denotes a connected component of this known part that contains a cell of the (upper) frontier. In the state, a component is represented by its frontier cells, which carry a common label. A *run* is a maximal block of consecutive positions that hold frontier cells. Since the cells of a row occupy only every other position, and the cells in between have already left the frontier, most runs consist of a single cell; only at the kink, the positions i and $i+1$ can both hold a frontier cell, and these two cells are neighbors. The frontier cells of a component form one or more runs. (In Figure 2, the two cells of component A are two runs, and component B has one run of two cells.) Since the components are separated by the frontier line from the undecided region, two components cannot *cross*: the pattern $A \ldots B \ldots A \ldots B$ does not occur.

The image $\rho(X)$ of a component X is again a connected component of the known part, but it need not touch the upper frontier. There are three possibilities:

- $\rho(X) = X$. We call X the *central* component. It is point-symmetric around the center, and it reaches the lower frontier. There is at most one such component.
- $\rho(X) = Y$ is another component. Then X and Y are *paired* components, and each is the *partner* of the other. A paired component reaches through the known part to the opposite (lower) frontier.
- $\rho(X)$ has no cell on the frontier. Then X is a *floating* component: it lives on the upper side only and does not reach the lower frontier, and its image touches only the lower frontier.

When two components X and Y are joined by a new cell, their images are joined as well, by the image of that cell. This is why the type of each component and the partner relation must be part of the state: if X and Y happen to be partners, the result is the central component; otherwise the new component $X \cup Y$ has the image $\rho(X) \cup \rho(Y)$.

The polyomino P is connected if and only if, at the end, there is a single component and it is the central component. Figure 3 shows an (artificial) example with all three types of components.

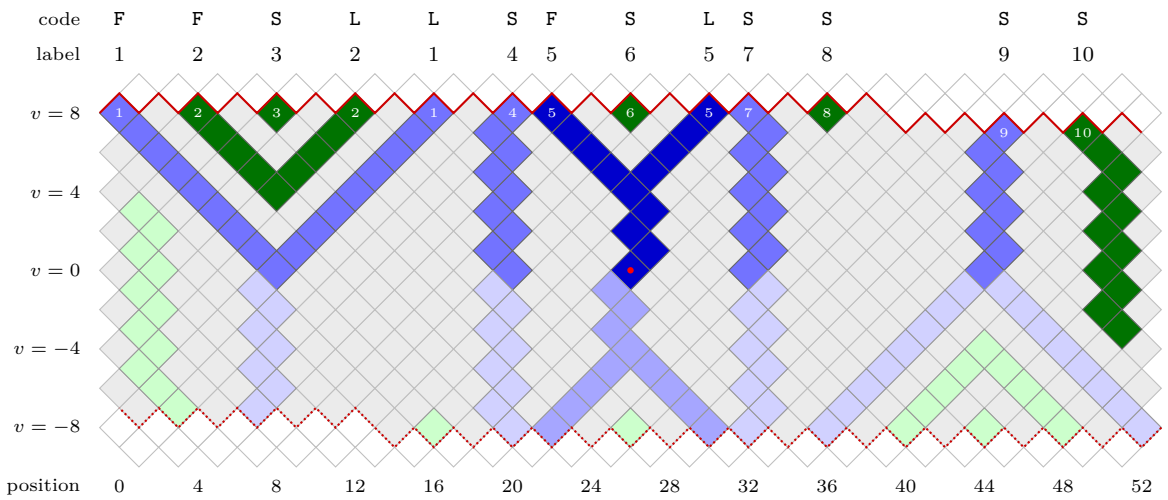


Figure 3: A larger example with all types of components: $W = 53$, center type C, row 8 has been decided up to position 38. The red line is the frontier, and the dotted line is its image, the lower frontier. The numbers are the component labels. The components 1 and 9 are *paired*, and so are 4 and 7 (blue): each of them reaches from the upper to the lower frontier, and its image is its partner. Component 5 is the *central* component (dark blue). The components 2, 3, 6, 8, and 10 are *floating* (green): their images (light green) touch only the lower frontier. Component 3 lies in the pocket between the two runs of the floating component 2, which in turn lies in the pocket of the paired component 1. Component 6 lies in the pocket of the central component. The components 8 and 10 are at the top level. Component 10 extends far into the lower half, but it does not reach the lower frontier. The crossing components form the nested chain $1, 4, 5, 7, 9$. Above the picture: the labels and the codes of the packed key (Section 3) at the occupied positions; all other positions have label 0 and code `E`.

¶ In addition, the state contains one bit that tells whether a side of the strip has already been *touched*, i.e., whether the leftmost or rightmost column contains a cell.

For each state we store how many different sets of decided cells lead to it, separately for each weight (number of cells of P so far).

2.2 The unpacked state

In its unpacked form, a state is a record of type *state_t* with the following fields:

- *label*$[i]$, $0 \le i < W$: the component label of the frontier cell at position i, or 0 if there is none. Labels are numbered $1, 2, \ldots,$ *num_components* in the order of their first appearance from left to right, which makes the representation canonical.
- *partner*$[j]$, for each component $j = 1, 2, \ldots$: the label of its partner if it is a paired component; j itself if j is the central component (it is its own image); and 0 if j is a floating component.
- *touches*: the *touched* bit.

A frontier contains at most $\lceil W/2 \rceil + 1$ cells, hence at most `MAX_COMPONENTS` components.

Figure 4 shows an example. The labels at the 17 positions are given above the picture. The partners for the components $1, 2, 3, 4$ are 4, 2, 0, 1, and the touched bit is 1, because the columns $u = -8$ and $u = 8$ are occupied.

13 ⟨ Type definitions 13 ⟩ ≡

```
typedef struct {
  uint8_t label[MAXW];      /* the component label at each position; 0 means no cell */
  uint8_t partner[MAXLAB];  /* for each component label, starting from 1 */
  uint8_t num_components;
  uint8_t touches;          /* the touched bit */
} state_t;
```

See also chunks 19, 24, 27, 33, and 37

This code is used in chunk 2.

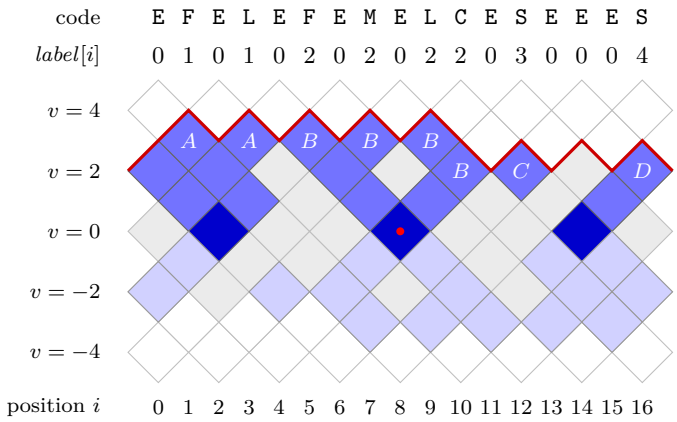


Figure 4: A state for the center type C with $W = 17$ columns, $u = -8\,..\,8$, after the cell $(u, v) = (1, 3)$ at position $i = 9$ has been decided. The upper half P^+ has 3 cells in the middle row (dark) and 16 other cells (blue), so the weight is $3 + 2 \cdot 16 = 35$. The images are light blue. There are four components. Component A (label 1) reaches down through the middle row to the image of the right arm; its image is the component D (label 4), so A and D are paired. Component B (label 2) contains the center: it is the central component. Component C (label 3) is an isolated cell in row 2; its image is not on the frontier: it is floating. Above the picture: the labels of the state, and the codes of the packed key (Section 3), abbreviated by their initials: `Empty`, `Continue`, `Singleton`, `First`, `Middle`, `Last`.

14 ¶ ⟨ Global variables 8 ⟩ +≡
```
static int MAX_COMPONENTS;      /* maximum number of components in a state */
```

¶ Two states are compared lexicographically. (If the labels agree, the number of components is the same.)

15 ⟨ Subroutines 9 ⟩ +≡
```
static int compare_states(const state_t *a, const state_t *b)
{    /* on entry: both states are canonical; only label[0..W − 1] and partner[1..num_components] are
        defined */
  int c ← memcmp(a→label, b→label, W);
  if (c) return c;
  c ← memcmp(a→partner + 1, b→partner + 1, a→num_components);
  if (c) return c;
  return a→touches − b→touches;
}
```

2.3 The working state

A transition changes only little: a new cell joins at most two components, and at most two cells leave the frontier. We therefore work directly on a copy of the state, with two arrays: $label[i]$ is the label of the frontier cell at position i (or 0), and $partner[j]$ is the partner of the component with label j, in the same form as in **state_t**. During a transition the labels need not be numbered consecutively, and there may be one label more than in a canonical key. All operations are simple sweeps over the positions.

16 **#define** `MAXLAB` (`MAXW` + 2) /* bound on the labels in the working arrays */

⟨ Subroutines 9 ⟩ +≡
```
static inline int is_central(const uint8_t *partner, int a)
{ return partner[a] ≡ a; }
static inline int is_floating(const uint8_t *partner, int a)
{ return partner[a] ≡ 0; }
static inline int is_paired(const uint8_t *partner, int a)
{ return partner[a] ≠ 0 ∧ partner[a] ≠ a; }
```

```
static inline void relabel(uint8_t *label, int from, int to)
{
  for (int j ← 0; j < W; j++)
    if (label[j] ≡ from) label[j] ← to;
}
```

2.4 Joining two components

When two components A and B become connected, their images $\rho(A)$ and $\rho(B)$ become connected as well. The cells of B get the label of A, and the type of the union depends on the types of A and B:

- If A and B are partners, the union is its own image: it becomes the central component.
- If one of the two is the central component, the union is the central component, and it swallows the partner of the other component if that component is paired.
- If both are paired (with different partners), their partners are joined, too, and the result is the partner of the union.
- If one is paired and the other is floating, the union is paired, with the same partner. (The partner has absorbed the image of the floating component.)
- If both are floating, so is the union.

17 ⟨ Subroutines 9 ⟩ +≡

```
static void join(uint8_t *label, uint8_t *partner, int a, int b)
{     /* on entry: a and b are labels that occur on the frontier, possibly a ≡ b; the partner relation
         is symmetric: partner[partner[x]] ≡ x for every component x that occurs and is not floating;
         afterwards b (and possibly a partner) no longer occurs, and its entry in partner is garbage */
  if (a ≡ b)      /* the new cell touches the same component a second time. */
    return;
  int partner_a ← partner[a], partner_b ← partner[b];
  relabel(label, b, a);
  if (partner_a ≡ b) partner[a] ← a;      /* a new central component has been formed */
  else if (partner_a ≡ a ∨ partner_b ≡ b) {      /* one of the components is central */
    if (partner_a ≠ 0 ∧ partner_a ≠ a) relabel(label, partner_a, a);
    if (partner_b ≠ 0 ∧ partner_b ≠ b) relabel(label, partner_b, a);
    partner[a] ← a;
  }
  else if (partner_a ≠ 0) {
    if (partner_b ≠ 0) relabel(label, partner_b, partner_a);
  }
  else if (partner_b ≠ 0) {
    partner[a] ← partner_b;
    partner[partner_b] ← a;
  }
}
```

¶ The canonical state is obtained by renumbering the components from left to right.

18 ⟨ Subroutines 9 ⟩ +≡

```
static void canonical_state(const uint8_t *label, const uint8_t *partner, int touches, state_t *result)
{     /* on entry: the labels are arbitrary numbers below MAXLAB, not necessarily consecutive; partner is
         valid for the labels that occur, and the partner of such a label occurs, too */
  uint8_t new_label[MAXLAB], old_label[MAXLAB];
  int num_components ← 0;
```

```
    memset(new_label, 0, sizeof (new_label));
    for (int i ← 0; i < W; i++) {
      if (label[i] ∧ ¬new_label[label[i]]) {
        new_label[label[i]] ← ++num_components;
        old_label[num_components] ← label[i];
      }
      result→label[i] ← new_label[label[i]];
    }
    for (int j ← 1; j ≤ num_components; j++) result→partner[j] ← new_label[partner[old_label[j]]];
    result→num_components ← num_components;
    result→touches ← touches;
  }
```

2.5 Removing a cell from the frontier

When the frontier cell at position i can no longer get new neighbors, it is *retired*. If other frontier cells of the same component remain, nothing happens. Otherwise the component X disappears from the frontier, and there are several cases:

- X is the central component. Then X is a complete symmetric polyomino, and nothing can be added to it. If the frontier is now empty, we have found a polyomino (`COMPLETE`); otherwise the other components can never be connected to X, and the state is `DEAD`.
- X is floating. Then X and $\rho(X)$ are two different pieces to which nothing can be added: `DEAD`.
- X is paired with a component Y. From now on, Y is floating.

19 ⟨ Type definitions 13 ⟩ +≡

```
  enum {
    OK, DEAD, COMPLETE
  };
```

20 ¶ ⟨ Subroutines 9 ⟩ +≡

```
  static int retire(uint8_t *label, uint8_t *partner, int i)
  {     /* on entry: the cell at position i has no undecided neighbor any more; the position may be empty
           */
    int retired_label ← label[i];
    if (¬retired_label) return OK;
    label[i] ← 0;
    int others_remain ← 0;
    for (int j ← 0; j < W; j++)
      if (label[j]) {
        others_remain ← 1;
        if (label[j] ≡ retired_label) return OK;
      }
    if (is_central(partner, retired_label)) return others_remain ? DEAD : COMPLETE;
    if (is_floating(partner, retired_label)) return DEAD;
    partner[partner[retired_label]] ← 0;      /* the partner is now floating */
    return OK;
  }
```

2.6 A transition

The function *step* takes a state *state* and decides the cell at position i of the current row: The flag *occupied* tells whether the cell belongs to P^+. The flag *last_in_row* is true for the last cell of the row. The result is `DEAD`, `COMPLETE`, or `OK`; in the last case the new canonical state is stored in $*result$.

A new cell gets a new component label and is floating at first, and it is joined with the frontier cells at positions $i-1$ and $i+1$. These are its two lower neighbors: at this moment, both positions hold cells of the previous row, see Figure 2. (The component that is given as the first argument of *join* keeps its label.)

After the decision, the cell at position $i-1$ has no undecided neighbors any more and is retired; at the end of a row, the same holds for position $i+1$. (Retiring an empty position does no harm.)

21 ⟨ Subroutines 9 ⟩ +≡

```
static int step(const state_t *state, int i, int occupied, int last_in_row, state_t *result)
{   /* on entry: state is canonical and not empty; label[i] ≡ 0, because the previous cell in this column,
       two rows below, has been retired; the cells of the current row are at positions ≤ i − 2, and the
       positions i − 1 and i + 1 hold cells of the previous row */
  uint8_t label[MAXW], partner[MAXLAB];
  memcpy(label, state→label, W);
  memcpy(partner, state→partner, state→num_components + 1);
  if (occupied) {
    int new_label ← state→num_components + 1;      /* the first unused label */
    partner[new_label] ← 0;
    label[i] ← new_label;
    for (int neighbor ← i − 1; neighbor ≤ i + 1; neighbor += 2)
      if (neighbor ≥ 0 ∧ neighbor < W ∧ label[neighbor]) join(label, partner, new_label, label[neighbor]);
  }
  int status ← OK;
  if (i − 1 ≥ 0) status ← retire(label, partner, i − 1);
  if (status ≡ OK ∧ last_in_row ∧ i + 1 < W) status ← retire(label, partner, i + 1);
  if (status ≠ OK) return status;
  canonical_state(label, partner, state→touches ∨ (occupied ∧ (i ≡ 0 ∨ i ≡ W − 1)), result);
  return OK;
}
```

3 Packed keys

The unpacked states are convenient but wasteful. In the tables, a state is stored as a bit string of `BYTES_PER_KEY` bytes:

- 3 bits for each position that can hold a frontier cell, with one of the following six codes 0–5: `C_EMPTY` for an empty position; `C_CONTINUE` if the cell continues the run of its left neighbor; and for the first cell of a run: `C_SINGLETON` if the component consists of this single run, `C_FIRST` for the first run of a component with several runs, `C_MIDDLE` for a middle run, `C_LAST` for the last run. Since components do not cross, a stack is sufficient to recover the labels, like for a sequence of parentheses.
- 1 bit per component: 0 if the component is floating, 1 otherwise.
- 1 bit: touched.

For the state of Figure 4, the codes are shown above the picture. Component 1 has two runs, at the positions 1 and 3 (`F`, `L`). Component 2 has three runs: at position 5 (`F`), at position 7 (`M`), and at the positions 9 and 10 (`L`, `C`), which are adjacent because the kink is between them. The components 3 and 4 are single cells (`S`). Only 9 of the 17 positions can hold a frontier cell: the positions $1, 3, 5, 7, 9$ of the current row and the positions $10, 12, 14, 16$ of the previous row. Their codes `FLFMLCSES` are followed by the bits $1, 1, 0, 1$ for the four components (only component 3 is floating) and by the touched bit 1. These are $3 \cdot 9 + 4 + 1 = 32$ bits, or 4 bytes.

3.1 The length of a key

Barequet and Ben-Shachar [1, 2] have shown that the number of valid sequences of the five codes other than `C_CONTINUE` for m frontier cells grows like $5^m m^{-3/2}$. Thus, $\log_2 5 \approx 2.32$ bits per frontier cell are necessary, and 3 bits per frontier cell are close to this information-theoretic minimum. Only every other position can hold a frontier cell: the frontier cells of a row occupy the positions of one parity. The only exception is the kink, where a cell of row v and a cell of row $v-1$ are in adjacent positions. (This is also the only place where the code `C_CONTINUE` can occur.) Thus there are at most $\lfloor W/2 \rfloor + 1$ frontier cells, and only their codes are stored. The key is about 1.3 times as long as the minimum. Ranking the states, as in [3], would give the shortest possible keys, but ranking and unranking are slow.

The length of the key is fixed for each strip. It is measured in bytes and not in larger units, in order to waste no space by rounding.

23 ⟨ Global variables 8 ⟩ +≡

```
static int BYTES_PER_KEY;      /* length of a packed key */
```

24 ¶ ⟨ Type definitions 13 ⟩ +≡
```
enum {
  C_EMPTY, C_CONTINUE, C_SINGLETON, C_FIRST, C_MIDDLE, C_LAST
};
```

3.2 One bit per component is enough

Call a component *crossing* if it is central or paired, that is, if it is not floating. The union of the frontier and its image under ρ is a closed curve around the known part of P (think of a disk), and a crossing component together with its partner connects the frontier with the image of the frontier across this disk. Two such connections cannot cross each other, and ρ reverses the order along the frontier. Consequently, the crossing components, read from left to right, form one nested chain

$$A_1,\ A_2,\ \ldots,\ A_k,\ (Z),\ \rho A_k,\ \ldots,\ \rho A_2,\ \rho A_1,$$

where the central component Z is present in the middle if and only if the number of crossing components is odd. In particular, there is at most one central component. Thus the j-th crossing component from the left is the partner of the j-th crossing component from the right, and it suffices to know *which* components are crossing. With the option `-C`, the program checks for every state that unpacking the packed key gives back the original state. We have run this check for $N \le 30$.

¶ Bit strings are written and read with two small routines. No field has more than 3 bits, so that a field extends over at most two bytes. Each bytes is filled from right to left.

26 ⟨ Subroutines 9 ⟩ +≡
```
static inline void put_bits(uint8_t *bytes, int *bit_position, int value, int num_bits)
{    /* on entry: the bytes have been cleared, and value has at most num_bits bits, num_bits ≤ 8 */
  int byte ← *bit_position ≫ 3, offset ← *bit_position & 7;
  bytes[byte] |= value ≪ offset;
  if (offset + num_bits > 8) bytes[byte + 1] |= value ≫ (8 − offset);
  *bit_position += num_bits;
}
static inline int get_bits(const uint8_t *bytes, int *bit_position, int num_bits)
{
  int byte ← *bit_position ≫ 3, offset ← *bit_position & 7;
  int value ← bytes[byte] ≫ offset;
  if (offset + num_bits > 8) value |= bytes[byte + 1] ≪ (8 − offset);
  *bit_position += num_bits;
  return value & ((1 ≪ num_bits) − 1);
}
```

3.3 Encoding and decoding

Packing and unpacking are done in two steps. The intermediate form is a *coded state*: the sequence of codes for the W positions, as a byte array, and one flag per component that tells whether the component is crossing. The first step translates between the labels and the codes; the second step is merely the compaction of the coded state into a bit string.

27 ⟨ Type definitions 13 ⟩ +≡
```
typedef struct {
  uint8_t code[MAXW];      /* C_EMPTY, ..., C_LAST */
  uint8_t crossing[MAXLAB];      /* for each component label, starting from 1 */
  uint8_t num_components;
  uint8_t touches;
} coded_t;
```

¶ Encoding. The first loop determines, for every component, the positions where its first and its last run start.

28 ⟨ Subroutines 9 ⟩ +≡

```
static void encode(const state_t *s, coded_t *coded)
{   /* on entry: the state is canonical: the first run of component j comes before the first run of
       component j + 1 */
  int first_run[MAXLAB], last_run[MAXLAB];
  int num_components ← 0;
  for (int i ← 0; i < W; i++)
    if (s→label[i] ∧ (i ≡ 0 ∨ ¬s→label[i − 1])) {
      int lab ← s→label[i];
      if (lab > num_components) {
        num_components ← lab;
        first_run[lab] ← i;
      }
      last_run[lab] ← i;
    }
  for (int i ← 0; i < W; i++) {
    int lab ← s→label[i];
    if (¬lab) coded→code[i] ← C_EMPTY;
    else if (i > 0 ∧ s→label[i − 1]) coded→code[i] ← C_CONTINUE;
    else if (first_run[lab] ≡ i) coded→code[i] ← last_run[lab] ≡ i ? C_SINGLETON : C_FIRST;
    else coded→code[i] ← last_run[lab] ≡ i ? C_LAST : C_MIDDLE;
  }
  for (int j ← 1; j ≤ num_components; j++) coded→crossing[j] ← s→partner[j] ≠ 0;
  coded→num_components ← num_components;
  coded→touches ← s→touches;
}
```

¶ Decoding. In the first part, *stack* holds the components with several runs whose last run has not yet been seen. In the second part, it holds the crossing components from left to right, and they are matched from the outside to the inside.

29 ⟨ Subroutines 9 ⟩ +≡

```
static void decode(const coded_t *coded, state_t *s)
{   /* on entry: the codes are properly nested, as produced by encode */
  int stack[MAXLAB], stack_size ← 0, num_components ← 0;
  for (int i ← 0; i < W; i++) {
    switch (coded→code[i]) {
    case C_EMPTY: s→label[i] ← 0;
      break;
    case C_CONTINUE: s→label[i] ← s→label[i − 1];
      break;
    case C_SINGLETON: s→label[i] ← ++num_components;
      break;
    case C_FIRST: s→label[i] ← ++num_components;
      stack[stack_size++] ← num_components;
      break;
    case C_MIDDLE: s→label[i] ← stack[stack_size − 1];
      break;
    case C_LAST: s→label[i] ← stack[−−stack_size];
      break;
    }
  }
  stack_size ← 0;   /* crossing components, left to right */
  for (int j ← 1; j ≤ num_components; j++)
    if (coded→crossing[j]) stack[stack_size++] ← j;
```

```
    else s→partner[j] ← 0;
  for (int a ← 0, b ← stack_size − 1; a ≤ b; a++, b−−) {
    s→partner[stack[a]] ← stack[b];      /* if a ≡ b, this is the central component */
    s→partner[stack[b]] ← stack[a];
  }
  s→num_components ← num_components;
  s→touches ← coded→touches;
}
```

3.4 Positions that can hold a frontier cell

Suppose that the cell at position *decided_position* was the last cell that has been decided. The cells of its row occupy the positions of the same parity, up to *decided_position*. The remaining frontier cells belong to the previous row: they are at the positions of the other parity, to the right of *decided_position*. (The cell at *decided_position* − 1 has just been retired.) All other positions are empty, and we don't store their codes. At the end of a row, and for the first row, *decided_position* is the last position of the row.

30 ⟨ Subroutines 9 ⟩ +≡

```
static inline int can_hold_frontier_cell(int j, int decided_position)
{
  return (((j ⊕ decided_position) & 1) ≠ 0) ≡ (j > decided_position);
}
```

3.5 Compaction into a bit string

This is the second step of packing, and its reverse. The number of components is not stored: it is the number of codes `C_SINGLETON` and `C_FIRST`.

31 ⟨ Subroutines 9 ⟩ +≡

```
static void compress(const coded_t *coded, int decided_position, uint8_t *bytes)
{    /* on entry: the positions that cannot hold a frontier cell have the code C_EMPTY */
  int pos ← 0;
  memset(bytes, 0, BYTES_PER_KEY);
  for (int i ← 0; i < W; i++)
    if (can_hold_frontier_cell(i, decided_position)) put_bits(bytes, &pos, coded→code[i], 3);
  for (int j ← 1; j ≤ coded→num_components; j++) put_bits(bytes, &pos, coded→crossing[j], 1);
  put_bits(bytes, &pos, coded→touches, 1);
}
static void expand(const uint8_t *bytes, int decided_position, coded_t *coded)
{
  int pos ← 0, m ← 0;
  for (int i ← 0; i < W; i++) {
    coded→code[i] ← can_hold_frontier_cell(i, decided_position) ? get_bits(bytes, &pos, 3) : C_EMPTY;
    if (coded→code[i] ≡ C_SINGLETON ∨ coded→code[i] ≡ C_FIRST) m++;
  }
  for (int j ← 1; j ≤ m; j++) coded→crossing[j] ← get_bits(bytes, &pos, 1);
  coded→num_components ← m;
  coded→touches ← get_bits(bytes, &pos, 1);
}
```

32 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void pack(const state_t *s, int decided_position, uint8_t *bytes)
{
  coded_t coded;
  encode(s, &coded);
  compress(&coded, decided_position, bytes);
}
```

```
static void unpack(const uint8_t *bytes, int decided_position, state_t *s)
{
  coded_t coded;
  expand(bytes, decided_position, &coded);
  decode(&coded, s);
}
```

4 Counters

The numbers that we count exceed 2^{64} at $n = 66$. There are two ways to deal with this. By default, the counters are unsigned 64-bit integers, and the arithmetic is modulo 2^{64}, or modulo a number $M < 2^{64}$ given by the option `-m`. The true values can then be recovered by the Chinese remainder theorem from two runs with different moduli, which should be relatively prime, for example 2^{64} and $2^{61} - 1$. (The script `tools/crt180.py` does this.) A third possibility, counters with fewer bits and more moduli, is discussed in Section 9.6. Alternatively, if the program is compiled with `-DCNT128`, the counters have 128 bits, and a single run gives the exact values. The records become twice as large, but for our runs with n up to 70, memory is not a bottleneck.

33 ⟨ Type definitions 13 ⟩ +≡

```
#ifdef CNT128
  typedef unsigned __int128 counter_t;      /* exact to 2^128 */
#define COUNTER_BITS "128"
#else
  typedef uint64_t counter_t;
#define COUNTER_BITS "64"
#endif
```

34 ¶ ⟨ Global variables 8 ⟩ +≡

```
static uint64_t MODULUS ← 0;      /* the modulus M; 0 means no reduction */
```

35 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static inline counter_t add_mod(counter_t a, counter_t b)
{     /* on entry: a and b are reduced modulo MODULUS; the sum may overflow if MODULUS > 2^63, which
         the test s < a detects */
  if (¬MODULUS) return a + b;
  counter_t s ← a + b;
  if (s < a ∨ s ≥ MODULUS) s −= MODULUS;
  return s;
}
```

¶ There is no `printf` format for 128-bit numbers. The function *counter_to_string* converts a counter to a decimal string. It uses eight buffers in rotation, so that several results can be used in the same call of `printf`.

36 ⟨ Subroutines 9 ⟩ +≡

```
static const char *counter_to_string(counter_t x)
{
  static char buffer[8][44];
  static int current_buffer;
  char *end ← buffer[current_buffer ← (current_buffer + 1) & 7] + 43, *digit ← end;
  *digit ← 0;
  do *−−digit ← '0' + (int)(x % 10), x /= 10; while (x);
  return digit;
}
```

5 The tables

The states of one stage of the computation are stored consecutively like in an array, and their indices in this array are stored in a hash table with open addressing (linear probing). The states are numbered $0, 1, \ldots, num_states - 1$ in the order of their creation; *keys* contains their packed keys, and the hash table *hash* contains the state numbers plus 1.

The counters of a state form a linked list of *records*, one record per weight for which the count is nonzero. This is much more compact than a full vector of $N + 1$ counters, because the weights of a state are confined to a narrow range: the pruning removes all weights that are too large, and at the peak there are only about two records per state. The list of a state is sorted by increasing weight. The weights in a list almost always form a contiguous range, in steps of 2 if the parity of the weight is fixed. This is the case for the centers of types M and V, where every weight is even. For type C, the parity of the weight tells whether the center cell is occupied, and about a third of the lists contain weights of both parities. (In a run with $N = 40$, the lists had 2.5 records on average and at most 28, and only 20 out of 9.5 million lists had a gap of more than 2 between successive weights. Each table was examined when it was complete, after every cell. At the end of a row, no list had such a gap.)

The counter records are stored consecutively in a storage area *records* like in an array. Its size is limited to $2^{32} - 1$ records.

37 ⟨ Type definitions 13 ⟩ +≡

```
typedef struct {      /* a counter record */
  counter_t count;
  uint32_t next;      /* index + 1; 0 means the end of the list */
  uint8_t weight;
} record_t;
typedef struct {
  size_t num_states, state_capacity, hash_mask;
  int decided_position;      /* determines the layout of the keys */
  uint8_t *keys;
  uint32_t *first_record;      /* index of the first record + 1 */
  record_t *records;
  size_t num_records, record_capacity;
  uint32_t *hash;      /* index of the state + 1; 0 means an empty slot */
  uint8_t *lowerbound;      /* the lower bound of each state */
} table_t;
```

38 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void *checked_realloc(void *p, size_t s)
{
  p ← realloc(p, s);
  if (¬p) {
    fprintf(stderr, "out␣of␣memory\n");
    exit(1);
  }
  return p;
}
```

39 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void table_allocate(table_t *t, size_t capacity)
{      /* on entry: BYTES_PER_KEY has been set for the current strip */
  t→num_states ← 0;
  t→state_capacity ← capacity;
  size_t hash_size ← 16;
  while (hash_size < 2 * capacity) hash_size ≪= 1;
  t→hash_mask ← hash_size − 1;
  t→keys ← checked_realloc(0, capacity * BYTES_PER_KEY);
  t→first_record ← checked_realloc(0, capacity * 4);
  t→hash ← 0;      /* allocated by table_initialize */
```

```
  t→lowerbound ← 0;      /* likewise */
  t→record_capacity ← 2 * capacity;
  t→num_records ← 0;
  t→records ← checked_realloc(0, t→record_capacity * sizeof(record_t));
}
static void table_free(table_t *t)
{
  free(t→keys);
  free(t→first_record);
  free(t→records);
  free(t→hash);
  free(t→lowerbound);
}
static size_t table_bytes(const table_t *t)
{
  return t→num_states * (BYTES_PER_KEY + 4) + t→num_records * sizeof(record_t) + (t→hash ?
      (t→hash_mask + 1) * 4 + t→num_states : 0);
}
```

5.1 The hash function

We use *universal hashing*, following Knuth's program POLYNUM [6], from which the function *hash_of_key* is taken (cf. *The Art of Computer Programming*, exercise 6.4–72): For each byte position of the key there is a table of 256 random numbers, and the hash value is the sum of the table entries for the bytes of the key. As Knuth remarks, this gives a good hash function although most of the key bits tend to be zero. The random numbers are produced at the start of the program by a generator of the SplitMix family [11, Section 3, Figure 16] with a fixed seed, so that all runs behave in the same way. In the terms of that figure, the state is advanced as in the method `nextSeed`, with the increment `GOLDEN_GAMMA` (the odd integer closest to $2^{64}/\phi$, which is the increment γ of a generator created with an explicit seed), and the new state is scrambled by the function `mix64variant13` (David Stafford's "Mix13" variant of the MurmurHash3 finalizer). We take the upper 32 bits of the result. The method `nextLong` of Figure 17 in [11] uses the function `mix64` in place of `mix64variant13`; the combination used here is the one that is widely known as "SplitMix64".

An earlier version of the program used an ad-hoc hash function (multiplications and shifts of the 64-bit words of the key). In a run with $N = 42$, a search in the hash table then looked at 1.02 occupied slots on average; with universal hashing this number is 0.44. (Three quarters of the searches are unsuccessful, and the tables are often much less than half full.)

The hash values have 32 bits. This is sufficient, because a table has fewer than 2^{32} states.

```
40 #define MAX_BYTES_PER_KEY 41      /* 3 · 81 + 81 + 1 = 325 bits for W = 160 */
⟨ Global variables 8 ⟩ +≡
  static uint32_t HASH_BITS[MAX_BYTES_PER_KEY][256];      /* random bits for universal hashing */
```

```
41 ¶ ⟨ Subroutines 9 ⟩ +≡
  static inline uint32_t hash_of_key(const uint8_t *key)
  {
    uint32_t h ← HASH_BITS[0][key[0]];
    for (int l ← 1; l < BYTES_PER_KEY; l++) h += HASH_BITS[l][key[l]];
    return h;
  }
```

¶ We can keep `HASH_BITS`$[l][0] \leftarrow 0$ without loss of universality.

```
42 ⟨ Initialize the random bits for hashing 42 ⟩ ≡
  {
    uint64_t x ← 0;      /* the state ("seed") of the generator */
    for (int l ← 0; l < MAX_BYTES_PER_KEY; l++)
      for (int b ← 1; b < 256; b++) {
        uint64_t z ← (x += #9E3779B97F4A7C15ULL);      /* nextSeed with GOLDEN_GAMMA */
```

```
        z ← (z ⊕ (z ≫ 30)) * #BF58476D1CE4E5B9ULL;      /* mix64variant13 */
        z ← (z ⊕ (z ≫ 27)) * #94D049BB133111EBULL;
        HASH_BITS[l][b] ← (z ⊕ (z ≫ 31)) ≫ 32;      /* upper half */
      }
  }
```

This code is used in chunk 101.

5.2 Growing, initializing, and finishing a table

When the table is full, the capacity is doubled and all states are rehashed. "Full" refers to the arrays *keys* and *first_record* of the states. The array *hash* of indices, in which the probing takes place, always has at least twice as many slots as there are states: its load factor lies between 1/4 and 1/2, and linear probing is fast in this range. This happens inside the function *table_create* below.

43 ⟨ Double the capacity of table t and rehash all states 43 ⟩ ≡

```
size_t capacity ← t→state_capacity * 2;
t→keys ← checked_realloc(t→keys, capacity * BYTES_PER_KEY);
t→first_record ← checked_realloc(t→first_record, capacity * 4);
t→lowerbound ← checked_realloc(t→lowerbound, capacity);
size_t hash_size ← (t→hash_mask + 1) * 2;
free(t→hash);
t→hash ← calloc(hash_size, sizeof(uint32_t));
if (¬t→hash) {
  fprintf(stderr, "out of memory\n");
  exit(1);
}
t→hash_mask ← hash_size − 1;
t→state_capacity ← capacity;
for (size_t state ← 0; state < t→num_states; state++) {
  size_t state_slot ← hash_of_key(t→keys + state * BYTES_PER_KEY) & t→hash_mask;
  while (t→hash[state_slot]) state_slot ← (state_slot + 1) & t→hash_mask;
  t→hash[state_slot] ← state + 1;
}
```

This code is used in chunk 45.

¶ The array *hash* is needed only while a table is being built. When the table is complete, its states are just scanned sequentially. The same holds for the array *lowerbound*, in which we remember the lower bound of each state (Section 6), so that it is computed only once and not every time when the state is reached. Therefore *table_initialize*, which prepares an empty table, allocates these two arrays, and *table_finish* gives them back as soon as the table is complete. In this way, only one of the two tables has these arrays at any time. (When *table_initialize* is called, the table is new or it has been finished, so that there is nothing to free.)

44 ⟨ Subroutines 9 ⟩ +≡

```
static void table_initialize(table_t *t, int decided_position)
{    /* on entry: the table is new or finished: it has no hash array */
  t→decided_position ← decided_position;
  t→num_states ← 0;
  t→num_records ← 0;
  t→hash ← calloc(t→hash_mask + 1, sizeof(uint32_t));      /* all slots empty */
  if (¬t→hash) {
    fprintf(stderr, "out of memory\n");
    exit(1);
  }
  t→lowerbound ← checked_realloc(0, t→state_capacity);
}
```

```
static void table_finish(table_t *t)
{
  free(t→hash);
  t→hash ← 0;
  free(t→lowerbound);
  t→lowerbound ← 0;
}
```

5.3 Finding and creating states

A state is looked up by its packed key. The function *table_find* returns the number of the state, or `NO_STATE` if the state is not in the table. In the latter case, *table_create* can be used to create the state, with an empty list of records. (The variable `HASH_SLOT` remembers the empty slot where the search of *table_find* has ended, so that *table_create* does not have to search again.)

45
```
#define NO_STATE ((size_t) −1)
⟨ Subroutines 9 ⟩ +≡
  static size_t HASH_SLOT;
  static size_t table_find(const table_t *t, const uint8_t *packed_key)
  {  /* on entry: the table is being built; unused bits of packed_key are zero, so that equal states have
        equal keys */
    size_t slot ← hash_of_key(packed_key) & t→hash_mask;
    while (t→hash[slot]) {
      size_t i ← t→hash[slot] − 1;
      if (¬memcmp(t→keys + i * BYTES_PER_KEY, packed_key, BYTES_PER_KEY)) return i;
      slot ← (slot + 1) & t→hash_mask;
    }
    HASH_SLOT ← slot;
    return NO_STATE;
  }
  static size_t table_create(table_t *t, const uint8_t *packed_key)
  {  /* on entry: the preceding operation on this table was an unsuccessful table_find for the same key
        */
    size_t slot ← HASH_SLOT;
    if (t→num_states ≡ t→state_capacity) {
      ⟨ Double the capacity of table t and rehash all states 43 ⟩
      slot ← hash_of_key(packed_key) & t→hash_mask;
      while (t→hash[slot]) slot ← (slot + 1) & t→hash_mask;
    }
    size_t i ← t→num_states++;
    t→hash[slot] ← i + 1;
    memcpy(t→keys + i * BYTES_PER_KEY, packed_key, BYTES_PER_KEY);
    t→first_record[i] ← 0;
    t→lowerbound[i] ← 0;
    return i;
  }
```

¶ The function *pack_and_check* packs a state. With the option `-C`, it checks that unpacking gives the state back.

46
```
⟨ Subroutines 9 ⟩ +≡
  static void pack_and_check(const state_t *s, int decided_position, uint8_t *packed_key)
  {
    pack(s, decided_position, packed_key);
    if (CHECK_PACKING) {
      state_t unpacked;
```

```
    unpack(packed_key, decided_position, &unpacked);
    if (unpacked.num_components ≠ s→num_components ∨ compare_states(&unpacked, s)) {
      fprintf(stderr, "pack/unpack␣mismatch\n");
      exit(2);
    }
  }
}
```

¶ The function *table_find_or_create* returns the number of the state with a given unpacked form. If the state is not yet in the table, it is created.

47 ⟨ Subroutines 9 ⟩ +≡

```
static size_t table_find_or_create(table_t *t, const state_t *s)
{
  uint8_t packed_key[MAX_BYTES_PER_KEY];
  pack_and_check(s, t→decided_position, packed_key);
  size_t i ← table_find(t, packed_key);
  return i ≠ NO_STATE ? i : table_create(t, packed_key);
}
```

5.4 Count records

A new record with weight *weight* and count *count* is taken from the end of the array of records. The function returns its number plus 1, as it is stored in the links.

48 ⟨ Subroutines 9 ⟩ +≡

```
static uint32_t new_record(table_t *t, int weight, counter_t count, uint32_t next)
{   /* on entry: the array records may be moved: the caller must not keep pointers into it */
  if (t→num_records ≡ t→record_capacity) {
    if (t→record_capacity ≥ #FFFFFFFF_U) {
      fprintf(stderr, "too␣many␣count␣records␣for␣32-bit␣links\n");
      exit(1);
    }
    t→record_capacity ← min(2 * t→record_capacity, #FFFFFFFF_U);
    t→records ← checked_realloc(t→records, t→record_capacity * sizeof(record_t));
  }
  record_t *q ← &t→records[t→num_records++];
  q→count ← count;
  q→weight ← weight;
  q→next ← next;
  return t→num_records;
}
```

¶ Adding the counters of one state to another. The counters of the state *source_state* of the table *source*, shifted by the weight *added_weight* of the newly added cell, are added to the state *target_state* of the table *target*, but only for the weights up to *weight_limit*. (This is where the pruning takes place: the caller chooses *weight_limit* such that the larger weights cannot lead to a polyomino with at most N cells.) Since both lists are sorted by increasing weight, this is done like a merge, in one simultaneous pass over the two lists. The variable *t_index* is the current position in the target list, and *previous* is the record before it (0 at the beginning of the list). We use indices and not pointers, because the array of records may be moved when it grows. (The tables *source* and *target* are always different.)

49 ⟨ Subroutines 9 ⟩ +≡

```
static void add_list(table_t *target, size_t target_state, const table_t *source, size_t source_state, int
    added_weight, int weight_limit)
{   /* on entry: source is a finished table and target is the table that is being built; both lists are
    sorted by increasing weight */
  uint32_t previous ← 0, t_index ← target→first_record[target_state];
```

```
  for (uint32_t s_index ← source→first_record[source_state];
       s_index ∧ source→records[s_index − 1].weight ≤ weight_limit;
       s_index ← source→records[s_index − 1].next) {
    int weight ← source→records[s_index − 1].weight + added_weight;
    while (t_index ∧ target→records[t_index − 1].weight < weight) {
      previous ← t_index;
      t_index ← target→records[t_index − 1].next;
    }
    if (t_index ∧ target→records[t_index − 1].weight ≡ weight)
      target→records[t_index − 1].count ← add_mod(target→records[t_index − 1].count,
          source→records[s_index − 1].count);
    else {
      uint32_t new_one ← new_record(target, weight, source→records[s_index − 1].count, t_index);
      if (previous) target→records[previous − 1].next ← new_one;
      else target→first_record[target_state] ← new_one;
      previous ← new_one;
    }
  }
}
```

6 The lower bound on the number of cells necessary to reach the boundary

The key to the efficiency of the program is a good lower bound on the number of cells that must still be added to a state before it can become a complete polyomino that is as high as it is wide. A state whose weight plus this bound exceeds N is discarded.

All bounds in this chapter count cells of P^+; each such cell has weight 2.

¶ First, a state in which every component is floating can never be completed: whatever is attached to a component X is attached in mirror image to $\rho(X)$, which is out of reach, so X and $\rho(X)$ can never meet.

51 ⟨ Return −1 if all components of *state* are floating 51 ⟩ ≡

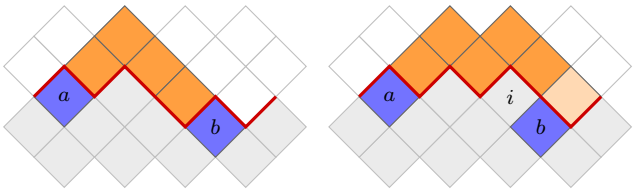
```
{
  int some_component_is_crossing ← 0;
  for (int j ← 1; j ≤ state→num_components; j++)
    if (state→partner[j] ≠ 0) some_component_is_crossing ← 1;
  if (¬some_component_is_crossing) return −1;
}
```

This code is used in chunk 73*.

6.1 Connections on the diagonal lattice

Suppose two frontier cells $a < b$ (we identify cells with their positions) in different runs have to be connected, and there are g empty positions between them. A path between them changes the column by ± 1 in every step, hence it needs at least one cell in each of the g columns strictly between a and b. Moreover, a connection between two consecutive runs and a connection between two other consecutive runs lie in disjoint sets of columns, so their costs simply add up. This is the great advantage of diagonal rows over horizontal rows: there, a single new cell can be adjacent to two frontier cells and to its successor in the row.

There is one exception, *at the kink* (Figure 5). Suppose position i has just been decided to be empty, and $b = i + 1$ is a frontier cell in row $v - 1$. The upper neighbors of b are the cell at position i of row v, which is empty, and the cell at position $i + 2$. Hence a path from the left must pass over b and come back to it from the right: it needs an additional cell in the column of b itself (and one in column $i + 2$, which we do not charge, because it might also be used for a connection to the right of b).



(a) $g = 4$ columns, 4 cells (b) at the kink: one more cell

Figure 5: Connecting two frontier cells a and b. (a) One cell per column between them. (b) The cell i left of b has been decided empty, and b is in the lower row: the path must cross the column of b above it. The last cell, drawn lighter, is not charged.

6.2 The cactus of gaps

Which gaps between consecutive runs have to be bridged? We must eventually connect all components, where a component and its partner count as the same object (if X is connected to Y then ρX is connected to ρY). Consider the graph whose nodes are the components, with partners identified, and with an edge for each gap between consecutive runs. Because components do not cross and because of the nested chain structure of the crossing components, this graph is a *cactus*: its cycles are edge-disjoint, and it can be decomposed as follows.

- *Pockets.* Between two consecutive runs of the same component X there may be components $C_1, \dots, C_r$ (at the outermost level inside these two runs). They can only be floating, and they appear nowhere else. The gaps X–C_1–$\cdots$–C_r–X form a cycle. To connect everything, all gaps of the cycle except one must be bridged, and the cheapest way is to omit the largest gap. The pockets inside each C_j are independent of this.
- *Outer paths.* At the top level, the components to the left of A_1 and to the right of ρA_1 hang on a path, and all their gaps must be bridged.
- *Rings.* The gaps from A_j to A_{j+1} on the left together with the gaps from ρA_{j+1} to ρA_j on the right form a cycle, since A_j and ρA_j are the same node. Again the largest gap can be omitted.
- *The middle.* The gaps from A_k over Z (if present) to ρA_k form the innermost cycle.

The sum of all gaps minus the largest gap of each cycle is the cost for connecting the cactus.

¶ **The central term.** Connecting the quotient is not enough: in the end, there must be a central component. If there is none yet, some component X must be joined to its own image. Closing a ring completely does not achieve this, because the right half of a ring is just the image of its left half. Only a path across the middle, from A_k to ρA_k, creates a central component. Therefore, if there is no central component, the middle cycle must be closed completely, and its largest gap is paid as well.

¶ The function *cycle_cost* handles one cycle with *num_gaps* gaps of sizes *gap*. It returns the sum minus the maximum. The remaining parameters and the calls of *reach_height* and *reach_score* have to do with the height, which is discussed next.

6.3 Reaching the sides and the top

The finished polyomino must touch the sides of the strip, and it must extend to row V^*, the last row for which the height reaches the width W. If no side has been touched yet, the frontier cell closest to a side needs a path of at least as many cells as there are columns between it and the side. In addition, each row above the current row up to row V^* must contain at least one cell. These requirements interact with the connections: a staircase to the side also climbs one row per cell, and the connecting paths between runs also reach some height.

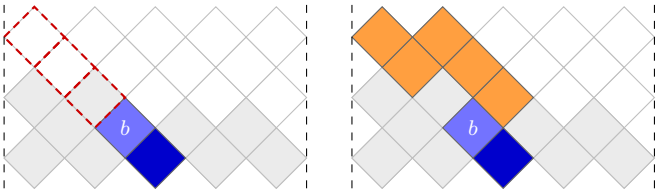
A path of c cells that connects frontier cells in rows r_a and r_b has its highest point at most in row $\lfloor (c + r_a + r_b + 1)/2 \rfloor$: it forms a "tent". The bound adds up the connection costs, determines the largest height h that the connecting paths (or the frontier itself) can reach, and charges one cell for each of the $V^* - h$ rows that are still missing.

There are choices. In a cycle, omitting the largest gap is cheapest, but bridging the largest gap and omitting the second largest one costs only the difference and may reach higher. If several gaps are tied for the largest, we may keep the higher ones for free. The bound takes the minimum over these alternatives. They are collected as *options* (extra cost, peak, score) in the arrays `OPTION_COST`, `OPTION_PEAK`, `OPTION_SCORE`.

¶ When no side has been touched yet, the situation is more delicate, because the path to the side may branch off from any cell of the structure, and it helps to reach the top at the same time. From a cell in row h and column j, a staircase that reaches both a side and row V^* needs

$$\max\bigl(V^* - h,\ \min(j,\, W - 1 - j)\bigr)$$

cells (Figure 6). We call the minimum of this quantity over all cells of the structure the *score*; the cells of the structure are the frontier cells and the cells of the connecting paths at their highest possible positions. The alternative is to reach the side and the top separately, at a cost of (distance to the side) + (missing rows).



(a) the bound: 3 cells (b) the truth: 5 cells

Figure 6: The center is in a cell, $W = 9$, $V^* = 4$. Rows 0 and 1 are complete, and in row 2 the first two cells have been decided empty. (a) The staircase bound for the cell b in row $h = 1$ and column $j = 3$ is $\max(4-1, 3) = 3$, but the first cell of the staircase is already known to be empty. (b) Every completion must use the right upper neighbor of b, and 5 cells are necessary. The lookahead at the kink finds this: it tries both possibilities for the next cell.

6.4 Functions for the height terms

The variables for the height computation are global, to keep the parameter lists short.

58 ⟨ Global variables 8 ⟩ +≡

```
static int CURRENT_ROW, TARGET_ROW, FIRST_ROW;      /* current row v, target row V*, first row v_0 */
static int FREE_HEIGHT, FREE_SCORE;      /* best height and best score reached for free */
static int NUM_OPTIONS, OPTION_COST[2*MAXW+4], OPTION_PEAK[2*MAXW+4], OPTION_SCORE[2*MAXW+4];
    /* options */
static int NUM_MIDDLE, MIDDLE_GAP[MAXW], MIDDLE_PEAK[MAXW], MIDDLE_SCORE[MAXW];
    /* the middle cycle: largest gap, its peak and score */
static int SIDE_DISTANCE;      /* distance to the nearest side, if untouched */
static int CURRENT_TOUCHES;      /* touched bit of the current state */
static int CURRENT_POSITION;      /* the position that was decided last */
```

¶ The row of the frontier cell at position j: The frontier cells up to the position that was decided last are in the current row, and the others are in the row below. (This function is called only for positions that hold a frontier cell.)

59 ⟨ Subroutines 9 ⟩ +≡

```
static inline int frontier_row(int j)
{      /* on entry: position j holds a frontier cell; CURRENT_ROW is the row that is being processed, and
          CURRENT_POSITION is the position that was decided last */
  return j ≤ CURRENT_POSITION ? CURRENT_ROW : CURRENT_ROW − 1;
}
```

60 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static inline void reach_height(int h) { FREE_HEIGHT ← max(FREE_HEIGHT, h); }
static inline void reach_score(int sc) { FREE_SCORE ← min(FREE_SCORE, sc); }
```

¶ The cost of reaching both the top row and a side from row h and column j.

61 ⟨ Subroutines 9 ⟩ +≡

```
static inline int side_and_top_cost(int h, int j)
{
  return max(TARGET_ROW − h, min(j, W − 1 − j));
}
```

¶ The function *gap_score* computes the best such cost over the cells of a path that bridges the gap between two frontier cells: the cell in column a and row *row_a*, and the cell in column b and row *row_b*. The path has a cell in every column j strictly between a and b. This cell is at most $j - a$ rows above *row_a*, and at most $b - j$ rows above *row_b*, because a path changes the row by one when it moves to the next column. We evaluate each of these cells at its highest possible row, which gives a lower bound on the cost.

At the kink, where the path must cross the column of the right frontier cell above this cell (Figure 5b), the caller passes $b + 2$ instead of b.

(This function could be streamlined: Check whether we are in the left half or in the right half; The best cell on a sequence of cells of slope ± 1 is the one closest to a diagonal line through the upper left or upper right corner; so one could directly determine which cell this is. But the total length of the loops in this function is of the order W, so it won't make a big difference.)

62 ⟨ Subroutines 9 ⟩ +≡

```
static int gap_score(int a, int row_a, int b, int row_b)
{    /* on entry: a < b */
  int best ← 1 ≪ 20;
  for (int j ← a + 1; j < b; j++) {
    int row_from_left ← row_a + (j − a), row_from_right ← row_b + (b − j);
    int c ← side_and_top_cost(min(row_from_left, row_from_right), j);
    best ← min(best, c);
  }
  return best;
}
```

¶ Now we can give the function *cycle_cost*. Besides returning the sum minus the maximum, it registers the peaks and scores of all gaps that are certainly bridged. If the largest gap *max1* is unique, bridging it instead of the second largest gap *max2* is recorded as an option with extra cost *max1* − *max2*. If the pointers are nonnull, the largest gap and its peak and score are returned (this is used for the central term).

63 ⟨ Subroutines 9 ⟩ +≡

```
static int cycle_cost(const int *gap, const int *peak, const int *score, int num_gaps, int
        *max1_out, int *peak_out, int *score_out)
{    /* on entry: num_gaps ≥ 1 */
  int sum ← 0, max1 ← −1, num_max ← 0, max2 ← 0, peak_of_max ← −1000, score_of_max ← 1 ≪ 20;
  for (int t ← 0; t < num_gaps; t++) {
    sum += gap[t];
    if (gap[t] > max1) {
      if (max1 ≥ 0) max2 ← max1;
      max1 ← gap[t];
      num_max ← 1;
      peak_of_max ← peak[t];
      score_of_max ← score[t];
    }
    else if (gap[t] ≡ max1) {
      num_max++;
      peak_of_max ← max(peak_of_max, peak[t]);
      score_of_max ← min(score_of_max, score[t]);
    }
    else if (gap[t] > max2) max2 ← gap[t];
  }
  for (int t ← 0; t < num_gaps; t++)
    if (num_max ≥ 2 ∨ gap[t] ≠ max1) {
      reach_height(peak[t]);
      reach_score(score[t]);
    }
  if (num_max ≡ 1) {
    OPTION_COST[NUM_OPTIONS] ← max1 − max2;
    OPTION_SCORE[NUM_OPTIONS] ← score_of_max;
    OPTION_PEAK[NUM_OPTIONS++] ← peak_of_max;
  }
```

```
  if (max1_out) {
    *max1_out ← max1;
    *peak_out ← peak_of_max;
    *score_out ← score_of_max;
  }
  return sum − max1;
}
```

¶* The function *finish_cost*(*height*, *score*) gives the number of cells that are still needed for reaching the top (and the side, if untouched) when the structure reaches height *height* and score *score*.

64 ⟨ Subroutines 9 ⟩ +≡

```
static inline int finish_cost(int height, int score)
{    /* on entry: the global variables of the height computation have been set by cactus_bound */
  int missing_rows ← max(TARGET_ROW − height, 0);
  if (CURRENT_TOUCHES) return missing_rows;
  int separately ← SIDE_DISTANCE + missing_rows;    /* edge and top separately */
  return min(score, separately);
}
```

¶ The function *height_term* takes the minimum over the options. At most one option is used at a time.

65 ⟨ Subroutines 9 ⟩ +≡

```
static int height_term(int height, int score)
{
  int extra ← finish_cost(height, score);
  for (int option ← 0; option < NUM_OPTIONS ∧ extra > 0; option++) {
    int extra_with_option ← OPTION_COST[option] + finish_cost(max(OPTION_PEAK[option], height),
        min(OPTION_SCORE[option], score));
    extra ← min(extra, extra_with_option);
  }
  return extra;
}
```

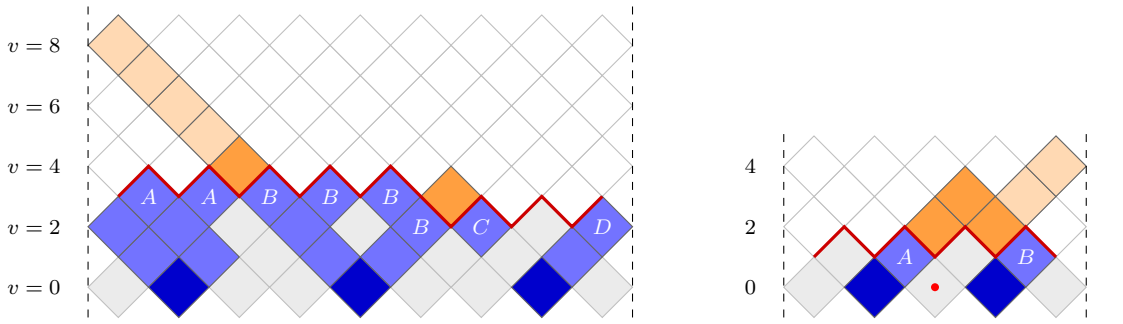
6.5 Examples

Figure 7 shows how the parts of the bound work together.

(a) This is the state of Figure 4. The runs of component B enclose two pockets without other components; they cost nothing. At the top level, the crossing components are A, B, D, where B is the central component and $D = \rho A$. The middle cycle leads from A over B and the floating component C to D, and its gaps have 1, 1, and 3 columns. The largest gap is omitted, and the other two are bridged with one cell each. There is no need to bridge the gap between C and D: when A is connected to B, the image of this connection connects $D = \rho A$ to $\rho B = B$. A central component exists and the sides are touched, so no further connections are needed. For the height, the polyomino must reach row $V^* = 8$. The frontier is in row 3, and the bridge between A and B is a tent of one cell, whose peak is in row 4. The rows $5, \ldots, 8$ need one cell each. The bound is therefore $2 + 4 = 6$ cells, or a weight of 12. With the weight 35 of the state, this state is kept if and only if $N \geq 47$. The bound is tight: the picture shows a completion with 6 cells.

(b) Here $W = 9$ and $V^* = 4$, and row 1 has just been finished. The middle row contains the cells $u = \pm 2$, but not the center. The two frontier cells form the components A and ρA. In the cactus, they are the same node, and connecting the cactus costs nothing. But there is no central component, and therefore the single gap of the middle cycle, with 3 columns, must be bridged. The tent over this gap can reach row 3. No side has been touched. The best cell of the structure for reaching both the right side and the top is the cell of the tent in row $h = 2$ at position $j = 6$: its score is $\max(V^* - h, \min(j, W - 1 - j)) = \max(2, 2) = 2$. The bound is $3 + 2 = 5$ cells, a weight of 10, and again there is a completion with this weight. It is a polyomino with $6 + 10 = 16$ cells.

Figure 8 shows four more examples with a central component Z and a floating component F, where no side has been touched. The center is in a cell. In the first three examples, Z reaches the frontier in two places, with F in the pocket between them, and row 2 is being processed.



(a) 6 cells: two bridges and four rows

(b) 5 cells: central term and staircase

Figure 7: Two states and cheapest completions. The cells that bridge gaps are orange, and the chain that leads to the top (and to the side) is light orange. Only the upper half is shown; the image of every new cell is added as well. (a) The state of Figure 4: the gaps A–B and B–C are bridged, the largest gap C–D of the middle cycle is omitted, and a staircase leads to row 8. (b) The components A and $B = \rho A$ are paired, and there is no central component: the gap between them must be bridged, and a staircase from the tent reaches the right side and row 4 simultaneously.

(a) $W = 17$, $V^* = 8$; the frontier cells are at the positions 2, 8, and 13. The pocket has gaps of 5 and 4 columns, but the right gap is at the kink and costs 5 cells as well. One of the gaps is omitted, and the hook over the left gap costs 5 cells. Its second cell is at position $j = 4$ in row $h = 4$, and the score of this cell is $\max(8-4, \min(4, 12)) = 4$: a straight chain of 4 cells leads into the corner, to column 0 in row 8. The bound is $5 + 4 = 9$ cells. (Reaching the side and the top separately would cost $5 + 2 + 3 = 10$.)

(b) The same, but the floating component is at position 4. One cell bridges the gap of one column. This cell is at position $j = 3$ in row $h = 3$, and its score is $\max(8 - 3, \min(3, 13)) = 5$. Now the missing rows dominate: the chain reaches the side after 3 cells, in row 6, bounces off, and needs 2 more cells for the rows 7 and 8. The bound is $1 + 5 = 6$ cells.

(c) $W = 19$, $V^* = 9$; the frontier cells are at the positions 7, 11, and 16. The gaps have 3 and 4 columns. Bridging the smaller gap costs 3 cells, but its cells are far from both sides: the best score is 8, for a total of 11. This is a case where an *option* wins: bridging the larger gap costs one cell more, and its highest cell, at position $j = 13$ in row $h = 4$, has score $\max(9 - 4, \min(13, 5)) = 5$, with a straight chain into the right corner. The bound is $4 + 5 = 9$ cells. (Reaching the side and the top separately would cost $3 + 2 + 5 = 10$.)

(d) $W = 17$, $V^* = 8$, and row 4 is being processed; the frontier cells are Z at position 6 and F at position 8. One cell bridges the gap. The best score is that of the frontier cell Z at position $j = 6$ in row $h = 4$: $\max(8 - 4, \min(6, 10)) = 6$. Here the distance to the side dominates: the chain reaches the top row after 4 cells, at position 2, and needs 2 more cells to arrive at the side. The bound is $1 + 6 = 7$ cells.

In all four cases, the picture shows a completion with this number of cells, so the bound is tight.

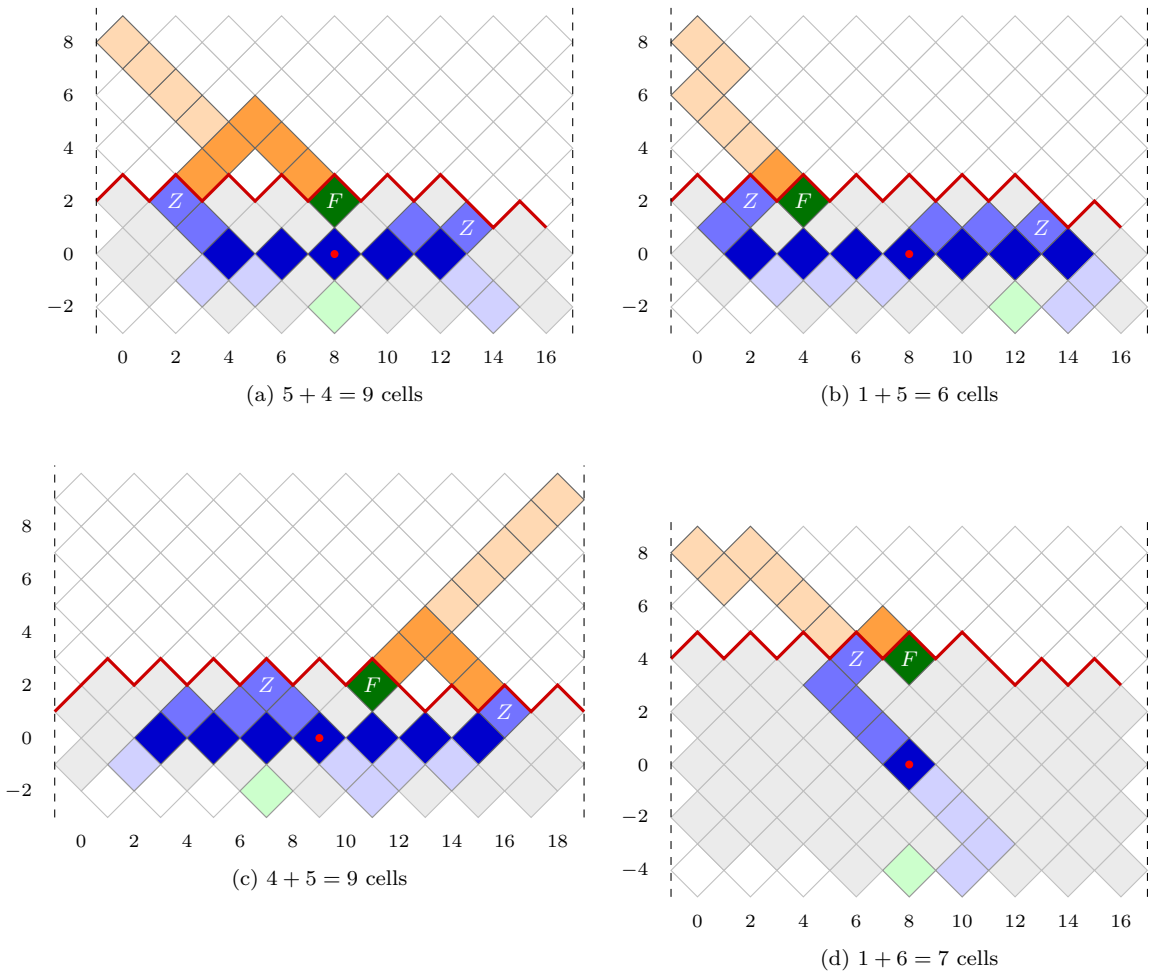
For the large state of Figure 3, the bound is 35 cells. The pockets of the components 1, 2, and 5 have two gaps of 3 columns each, of which one must be bridged ($3 + 3 + 3$ cells). The ring formed by the components 1 and 4 on the left and 7, 8, and 9 on the right has gaps of 3, 3, and 8 columns; the largest is omitted (6 cells). The middle cycle through the components 4, 5, and 7 has two gaps of one column (1 cell). The floating component 10 hangs on the outer path behind component 9, with a gap of 3 columns (3 cells). These are 19 cells. A tent over a gap of 3 columns in row 8 reaches row 10, and the top row is $V^* = 26$, so 16 more cells are needed for the missing rows. The values have been confirmed by calling the function *bound* of the program for these states.

6.6 Putting the bound together

The parameter *kink* is the position of the frontier cell right of the kink, if the cell before it has been decided empty, and -1 otherwise.

67 ⟨ Subroutines 9 ⟩ +≡

```
static int cactus_bound(const state_t *state, int kink)
{    /* on entry: CURRENT_ROW and TARGET_ROW have been set; kink is −1, or the position right of the
        kink if that position holds a frontier cell and the position left of it does not */
  const uint8_t *label ← state→label;
  int touches ← state→touches;
```



(a) $5 + 4 = 9$ cells (b) $1 + 5 = 6$ cells

(c) $4 + 5 = 9$ cells (d) $1 + 6 = 7$ cells

Figure 8: Four states with a central component Z and a floating component F (green), and cheapest completions: hooks that bridge a gap (orange) and the chain to the side and to the top (light orange). The images in the lower half are drawn only for the cells of the state, not for the cells of the completion. (a) The chain leads straight into the corner. (b) The chain bounces off the side and continues to the top. (c) It is cheaper to bridge the larger gap, because the chain from there is shorter. (d) The chain reaches the top first and continues along the top to the side.

```
    ⟨ Find the runs of the frontier; if the frontier is empty, return 0 68 ⟩
    ⟨ Initialize the height computation with the frontier cells 69 ⟩
    int cost ← 0;
    ⟨ Scan the runs from left to right and pay for the pockets 70 ⟩
    ⟨ Pay for the outer paths, the rings and the middle 71 ⟩
    ⟨ Add the central term and the terms for the side and the top 72 ⟩
    return cost;
}
```

¶ Run j extends from position $run_start[j]$ to $run_end[j]$ and belongs to component $run_label[j]$.

68 ⟨ Find the runs of the frontier; if the frontier is empty, return 0 68 ⟩ ≡

```
int run_start[MAXW], run_end[MAXW], run_label[MAXW], num_runs ← 0;
int last_run[MAXW + 2];
for (int i ← 0; i < W; i++)
  if (label[i]) {
    if (i ≡ 0 ∨ ¬label[i − 1]) {
      run_start[num_runs] ← i;
      run_label[num_runs] ← label[i];
      num_runs++;
```

```
    }
    run_end[num_runs − 1] ← i;
    last_run[label[i]] ← num_runs − 1;
  }
if (¬num_runs) return 0;
```

This code is used in chunk 67.

69 ¶ ⟨ Initialize the height computation with the frontier cells 69 ⟩ ≡

```
FREE_HEIGHT ← −1000;
FREE_SCORE ← 1 ≪ 20;
NUM_OPTIONS ← 0;
NUM_MIDDLE ← 0;
CURRENT_TOUCHES ← touches;
for (int i ← 0; i < W; i++)
  if (label[i]) {
    reach_height(frontier_row(i));
    reach_score(side_and_top_cost(frontier_row(i), i));
  }
```

This code is used in chunk 67.

¶ The pockets are processed with a stack, like nested parentheses. The gaps of the open pockets are collected in a buffer (*pocket_gap*, with peaks *pocket_peak* and scores *pocket_score*); *pocket_label* and *pocket_first_gap* give the component of each open pocket and the start of its gaps in the buffer. Gaps at the top level are stored separately: *top_gap*[*t*] is the gap after the *t*-th top-level component *top_label*[*t*]. When a run of the component on top of the stack is found, the pocket is closed and paid for.

70 ⟨ Scan the runs from left to right and pay for the pockets 70 ⟩ ≡

```
    /* open pockets: component and start of its gaps in the gap buffer */
int pocket_label[MAXW], pocket_first_gap[MAXW], num_open_pockets ← 0;
int pocket_gap[MAXW], pocket_peak[MAXW], pocket_score[MAXW], num_pocket_gaps ← 0;
int top_label[MAXW], top_gap[MAXW], top_peak[MAXW], top_score[MAXW], num_top ← 0;
for (int j ← 0; j < num_runs; j++) {
  int component ← run_label[j];
  if (j > 0) {
    int a ← run_end[j − 1], b ← run_start[j];
    int gap ← b − a − 1 + (b ≡ kink);      /* at the kink the path ends one column beyond b: g + 2 cells */
    int peak ← (gap − 1 + (b ≡ kink) + frontier_row(a) + frontier_row(b) + 2)/2;
    int score ← touches ? 1 ≪ 20 : gap_score(a, frontier_row(a), b ≡ kink ? b + 2 : b, frontier_row(b));
    if (num_open_pockets) {
      pocket_gap[num_pocket_gaps] ← gap;
      pocket_score[num_pocket_gaps] ← score;
      pocket_peak[num_pocket_gaps++] ← peak;
    }
    else {
      top_gap[num_top − 1] ← gap;
      top_score[num_top − 1] ← score;
      top_peak[num_top − 1] ← peak;
    }
  }
  if (num_open_pockets ∧ pocket_label[num_open_pockets − 1] ≡ component) {
    num_open_pockets−−;
    cost += cycle_cost(pocket_gap + pocket_first_gap[num_open_pockets],
        pocket_peak + pocket_first_gap[num_open_pockets],
        pocket_score + pocket_first_gap[num_open_pockets],
        num_pocket_gaps − pocket_first_gap[num_open_pockets], 0, 0, 0);
    num_pocket_gaps ← pocket_first_gap[num_open_pockets];
  }
  else if (¬num_open_pockets) top_label[num_top++] ← component;
```

```
    if (j < last_run[component]) {
      pocket_label[num_open_pockets] ← component;
      pocket_first_gap[num_open_pockets] ← num_pocket_gaps;
      num_open_pockets++;
    }
  }
```

This code is used in chunk 67.

¶ At the top level, *crossing* lists the m crossing components. Ring number i consists of the gaps between *crossing*$[i]$ and *crossing*$[i+1]$ and their counterparts on the right; the last "ring", $i = k-1$, is the middle.

71 ⟨ Pay for the outer paths, the rings and the middle 71 ⟩ ≡ /* top level */

```
  int crossing[MAXW], num_crossing ← 0;
  for (int t ← 0; t < num_top; t++)
    if (state→partner[top_label[t]] ≠ 0)      /* crossing */
      crossing[num_crossing++] ← t;
  if (num_crossing) {
    for (int t ← 0; t < crossing[0]; t++) {
      cost += top_gap[t];
      reach_height(top_peak[t]);
      reach_score(top_score[t]);
    }
    for (int t ← crossing[num_crossing − 1]; t < num_top − 1; t++) {
      cost += top_gap[t];
      reach_height(top_peak[t]);
      reach_score(top_score[t]);
    }
    int num_pairs ← num_crossing/2;
    NUM_MIDDLE ← 0;
    for (int i ← 0; i < num_pairs; i++) {
      int gap[2 * MAXW], peak[2 * MAXW], score[2 * MAXW], num_gaps ← 0, max1, peak_of_max, score_of_max;
      int a ← crossing[i], b ← (i < num_pairs − 1) ? crossing[i + 1] : crossing[num_crossing − 1 − i];
      for (int t ← a; t < b; t++) {
        gap[num_gaps] ← top_gap[t];
        score[num_gaps] ← top_score[t];
        peak[num_gaps++] ← top_peak[t];
      }
      if (i < num_pairs − 1)
        for (int t ← crossing[num_crossing − 2 − i]; t < crossing[num_crossing − 1 − i]; t++) {
          gap[num_gaps] ← top_gap[t];
          score[num_gaps] ← top_score[t];
          peak[num_gaps++] ← top_peak[t];
        }
      cost += cycle_cost(gap, peak, score, num_gaps, &max1, &peak_of_max, &score_of_max);
      if (i ≡ num_pairs − 1) {      /* only a path across the middle (from A_k to ρA_k) can join some X to
            ρX: the right side of a ring is just the image of its left side */
        MIDDLE_GAP[NUM_MIDDLE] ← max1;
        MIDDLE_SCORE[NUM_MIDDLE] ← score_of_max;
        MIDDLE_PEAK[NUM_MIDDLE++] ← peak_of_max;
      }
    }
    if (num_crossing & 1) NUM_MIDDLE ← 0;      /* central component present */
  }
  else {
    for (int t ← 0; t < num_top − 1; t++) {
      cost += top_gap[t];
      reach_height(top_peak[t]);
      reach_score(top_score[t]);
    }
```

```
}
```
This code is used in chunk 67.

72 ¶* ⟨ Add the central term and the terms for the side and the top 72 ⟩ ≡
```
SIDE_DISTANCE ← 0;
if (¬touches) {
  int left_distance ← run_start[0], right_distance ← W − 1 − run_end[num_runs − 1];
  SIDE_DISTANCE ← min(left_distance, right_distance);
}     /* without a central component the middle cycle must be closed completely: pay its largest gap,
         which then also counts for the height */
if (NUM_MIDDLE) {
  int extra ← 1 ≪ 20;
  for (int c ← 0; c < NUM_MIDDLE; c++) {
    int extra_this_way ← MIDDLE_GAP[c] + height_term(max(MIDDLE_PEAK[c], FREE_HEIGHT),
        min(MIDDLE_SCORE[c], FREE_SCORE));
    extra ← min(extra, extra_this_way);
  }
  cost += extra;
}
else cost += height_term(FREE_HEIGHT, FREE_SCORE);
```
This code is used in chunk 67.

6.7 The complete bound

The function *bound* returns a lower bound on the *weight* that must still be added to a nonempty state, or -1 if the state can never be completed. Here k is the number of the current row, counted from the first row of P^+, and *min_rows* is the smallest number of rows for which the height reaches W.

73 ⟨ Subroutines 9 ⟩ +≡
```
static int bound(const state_t *state, int k, int min_rows, int i, int last_in_row)
{     /* on entry: state is not empty; it is the state after position i of row k has been decided; FIRST_ROW
         has been set */
  const uint8_t *label ← state→label;
  int lowerbound ← max(2 * (min_rows − k), 0);
  if (¬state→touches) {
    int side_distance ← W;
    for (int j ← 0; j < W; j++)
      if (label[j]) {
        int distance ← min(j, W − 1 − j);
        side_distance ← min(side_distance, distance);
      }
    lowerbound ← max(lowerbound, 2 * side_distance);
  }
  ⟨ Return −1 if all components of state are floating 51 ⟩
  int kink ← (¬last_in_row ∧ ¬label[i] ∧ i + 1 < W ∧ label[i + 1]) ? i + 1 : −1;
  CURRENT_ROW ← FIRST_ROW + k − 1;
  CURRENT_POSITION ← i;
  TARGET_ROW ← FIRST_ROW + min_rows − 1;
  int connection_bound ← 2 * cactus_bound(state, kink);
  lowerbound ← max(lowerbound, connection_bound);
  return lowerbound;
}
```

7 The main loop

The results of a case are collected in two vectors, indexed by n: PORTRAIT for the polyominoes that are higher than wide ($uspan < vspan$) and SQUARE for $uspan = vspan$.

74 ⟨ Global variables 8 ⟩ +≡
```
static counter_t *ROW_RESULTS;      /* results for the current number of rows */
static counter_t *PORTRAIT, *SQUARE;      /* total results of one case */
static size_t MAX_STATES, MAX_BYTES, MAX_RECORDS, MAX_KEY_BYTES;      /* statistics */
```

¶ These vectors are allocated at the beginning of the program, as soon as N is known.

75 ⟨ Allocate the result vectors 75 ⟩ ≡
```
PORTRAIT ← calloc(NMAX_plus_1, sizeof(counter_t));      /* indices 0..N */
SQUARE ← calloc(NMAX_plus_1, sizeof(counter_t));
ROW_RESULTS ← calloc(NMAX_plus_1, sizeof(counter_t));
```
This code is used in chunk 102*.

¶ When a polyomino is complete after $k-1$ rows, it is counted if it touches the sides and is at least as high as wide. It is first counted in the vector `ROW_RESULTS`, which collects the polyominoes with the current number of rows. At the end of the row, this vector is added to `SQUARE` or `PORTRAIT`. At this point, the state *source_state* of the table *current* has just become a complete polyomino while row k is processed, so its top row has the number $k-1$.

76 ⟨ Record the polyominoes that are completed from state *source_state* 76 ⟩ ≡
```
if (old_state.touches ∧ vspan(k − 1) ≥ W)
  for (uint32_t r ← current→first_record[source_state]; r; r ← current→records[r − 1].next) {
    int weight ← current→records[r − 1].weight;
    ROW_RESULTS[weight] ← add_mod(ROW_RESULTS[weight], current→records[r − 1].count);
  }
```
This code is used in chunk 92.

7.1 Reporting

Bounding rectangles. The polyominoes of one case with a given number of rows have the same bounding rectangle in diagonal coordinates: W columns wide and *vspan*(*top_row_number*) rows high. All polyominoes with *top_row_number* rows are completed while row *top_row_number* + 1 is processed. At the end of that row, the function *print_row_results* writes one line to the log with the number of polyominoes in this rectangle for every n. (The vector `ROW_RESULTS` is cleared at the beginning of every row.) The line names the type of center, the width with the range of u, the height with the range of v, and then the nonzero counts in the form `n=`n`:`count. (The counts are modulo M, like everything else. There are no counts for $n > N$, because weights larger than N are pruned.) Only rectangles that are at least as high as wide are reported; the caller tests this. (A line without any counts would mean that no polyomino with at most N cells has this bounding rectangle, although some state survived the pruning up to this row.) The numbers for a rectangle that is wider than high are those of the transposed rectangle, with the same type of center, except that the vertical edge becomes a horizontal edge; and the case $(M_x, M_y) = (1, 0)$ of a vertical edge that we compute stands also for the case $(0, 1)$ of a horizontal edge with the same numbers.

The type of center is not determined by the size of the rectangle alone: if both sides are even, the center is the midpoint of an edge, but if both sides are odd, the center can be the center of a cell or a vertex of the grid.

77 ⟨ Subroutines 9 ⟩ +≡
```
static void print_row_results(int top_row_number)
{
  static const char *type_name[3] ← {"R180C␣center␣in␣a␣cell",
      "R180M␣center␣on␣the␣midpoint␣of␣a␣vertical␣edge", "R180V␣center␣at␣a␣vertex"};
  int height ← vspan(top_row_number);
  int top_row ← TWIN ? (M_v + 1)/2 + top_row_number − 1 : M_v/2 + top_row_number;
  if (VERBOSITY ≥ 1) {
    fprintf(stderr, "#rect␣%s␣(Mx=%d␣My=%d):␣width␣%d␣(u=%d..%d)␣x␣height␣%d␣(v=%d..%d):",
        type_name[M_x + M_y], M_x, M_y, W, u_min, u_min + W − 1, height, M_v − top_row, top_row);
    for (int n ← 0; n ≤ NMAX; n++)
      if (ROW_RESULTS[n]) fprintf(stderr, "␣n=%d:%s", n, counter_to_string(ROW_RESULTS[n]));
    fprintf(stderr, "\n");
```

```
    fflush(stderr);
  }
}
```

¶ Progress is reported on the standard error stream, after every row (`-v 1`, the default) or after every cell (`-v 2`). With `-v 1`, a line is also written within a row when more than `PROGRESS_SECONDS` seconds have passed since the last line. Every line is flushed immediately. The field "done" is the number of polyominoes that the current case has completed so far (modulo M).

78 ⟨ Global variables 8 ⟩ +≡

```
static int VERBOSITY ← 1;      /* option -v: 0 none, 1 per row, 2 per cell */
static double PROGRESS_SECONDS ← 60;
    /* option -p: within a row, a line at most every PROGRESS_SECONDS seconds */
static double START_TIME, LAST_REPORT_TIME;
```

79 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static double wall_time(void)
{
  struct timespec ts;
  clock_gettime(CLOCK_MONOTONIC, &ts);
  return ts.tv_sec + 1 · 10^-9 * ts.tv_nsec;
}
static void print_status_line(const char *tag, int k, int v, int cell_number, int num_cells, const
        table_t *current, const table_t *next)
{
  double now ← wall_time( );
  LAST_REPORT_TIME ← now;
  counter_t done ← 0;
  for (int n ← 0; n ≤ NMAX; n++)
    done ← add_mod(done, add_mod(add_mod(PORTRAIT[n], SQUARE[n]), ROW_RESULTS[n]));
  fprintf(stderr,
      "%s␣Mx=%d␣My=%d␣umax=%d␣W=%d␣row␣k=%d␣v=%d␣cell␣%d/%d:␣""%zu␣states,␣%zu␣rec\
      ords,␣%.1f␣MB,␣done␣%s,␣%.1fs␣wall,␣%.1fs␣cpu\n", tag, M_x, M_y, u_max, W, k, v, cell_number,
      num_cells, current→num_states, current→num_records,
      (table_bytes(current) + table_bytes(next))/1 · 10^6, counter_to_string(done), now − START_TIME,
      (double) clock( )/CLOCKS_PER_SEC);
  fflush(stderr);
}
```

7.2 Treating one case

Here is the function that treats one case.

80 ⟨ Subroutines 9 ⟩ +≡

```
static void count_case(void)
{    /* on entry: M_x, M_y, and u_max specify the case, and the result vectors have been allocated */
  ⟨ Set up the geometry of the case 82 ⟩
  ⟨ Local variables of count_case 83 ⟩
  ⟨ Create the initial states 84 ⟩
  int row_cells[MAXW];
  for (int k ← 1 + TWIN; current→num_states; k++) {
    memset(ROW_RESULTS, 0, NMAX_plus_1 * sizeof(counter_t));
    ⟨ Determine the cells of row k 90 ⟩
    for (int cell_number ← 0; cell_number < num_row_cells; cell_number++) {
      ⟨ Decide cell cell_number of the row for all states 91 ⟩
    }
    ⟨ Merge every state with its mirror image 96 ⟩
```

```
    if (VERBOSITY ≥ 1) print_status_line("#row␣", k, v, num_row_cells, num_row_cells, current, next);
    ⟨ Add the results of this row to the totals and report them 81 ⟩
  }
  table_free(&A);
  table_free(&B);
}
```

¶ The polyominoes that are completed while row k is processed have $k-1$ rows. They have been collected in ROW_RESULTS, provided that they are at least as high as wide; lower rectangles are not counted in this case.

81 ⟨ Add the results of this row to the totals and report them 81 ⟩ ≡

```
if (vspan(k − 1) ≡ W) {
  for (int n ← 0; n ≤ NMAX; n++) SQUARE[n] ← add_mod(SQUARE[n], ROW_RESULTS[n]);
  print_row_results(k − 1);
}
else if (vspan(k − 1) > W) {
  for (int n ← 0; n ≤ NMAX; n++) PORTRAIT[n] ← add_mod(PORTRAIT[n], ROW_RESULTS[n]);
  print_row_results(k − 1);
}
```

This code is used in chunk 80.

82 ¶⟨ Set up the geometry of the case 82 ⟩ ≡

```
M_v ← M_x + M_y;
M_u ← M_x − M_y;
u_min ← M_u − u_max;
W ← u_max − u_min + 1;
TWIN ← M_v & 1;
memset(PORTRAIT, 0, NMAX_plus_1 ∗ sizeof(counter_t));
memset(SQUARE, 0, NMAX_plus_1 ∗ sizeof(counter_t));
memset(ROW_RESULTS, 0, NMAX_plus_1 ∗ sizeof(counter_t));
if (W ≡ 1) {      /∗ The strip with a single column is a special case. ∗/
  if (M_x ≡ 0 ∧ M_y ≡ 0) {
    SQUARE[1] ← ROW_RESULTS[1] ← 1;
    print_row_results(0);
  }
  return;
}
if (W > MAXW) {
  fprintf(stderr, "strip␣too␣wide\n");
  exit(1);
}
MAX_COMPONENTS ← (W + 1)/2 + 1;
BYTES_PER_KEY ← (3 ∗ (W/2 + 1) + MAX_COMPONENTS + 1 + 7)/8;
NMAX_plus_1 ← NMAX + 1;
int min_rows ← 0;
while (vspan(min_rows) < W) min_rows++;
```

This code is used in chunk 80.

¶ Two tables are used alternately: *current* for the states before the current cell and *next* for the states after it.

83 ⟨ Local variables of *count_case* 83 ⟩ ≡

```
table_t A, B, ∗current ← &A, ∗next ← &B;
table_allocate(&A, 1024);
table_allocate(&B, 1024);
state_t new_state;
int first_row;
```

This code is used in chunk 80.

7.3 Initial states

The first row is not built cell by cell. We enumerate all possibilities for this row and create one state for each of them. The possibilities are the nonempty subsets of a list *choice* of *num_choices* positions. Since their number is exponential in W, we must not look at all of them: a subset with more than *max_chosen* elements is too heavy, whatever is added later, because $min_rows - k$ further rows are necessary in any case, with weight 2 each.

84 ⟨ Create the initial states 84 ⟩ ≡

```
int choice[MAXW], num_choices ← 0, max_chosen;
if (TWIN ≡ 0) {
  ⟨ Prepare the choices for the middle row 86 ⟩
}
else {
  ⟨ Prepare the choices for the twin row 88 ⟩
}
uint8_t chosen[MAXW];      /* the current subset */
int num_chosen ← 0, done ← 0;
memset(chosen, 0, num_choices);
table_initialize(current, choice[num_choices − 1]);      /* the last position of the first row */
while (¬done) {
  ⟨ Generate the next subset for which the size num_chosen does not exceed max_chosen; set done if
      finished 85 ⟩
  if (done) break;
  if (TWIN ≡ 0) {
    ⟨ Create the state for a subset of the middle row 87 ⟩
  }
  else {
    ⟨ Create the state for a subset of the twin row 89 ⟩
  }
}
table_finish(current);
```

This code is used in chunk 80.

¶ The subsets are generated in the order of binary numbers, with the array *chosen* as the counter, starting from the empty set. Normally, the next subset is obtained by adding 1: the lowest position that is not chosen becomes chosen, and all positions below it are cleared. This increases the size by at most 1. However, if the current subset has already the maximum size, we add 1 at its lowest chosen position j instead. This skips all subsets that consist of the current subset plus some positions below j. These are supersets of the current subset, and they are too large. The new subset is not larger than the current one. In this way, a subset that is too large is never generated.

85 ⟨ Generate the next subset for which the size num_chosen does not exceed max_chosen; set done if
finished 85 ⟩ ≡

```
{
  int j ← 0;
  if (num_chosen ≥ max_chosen)
    while (j < num_choices ∧ ¬chosen[j]) j++;      /* the lowest chosen position */
  while (j < num_choices ∧ chosen[j]) {      /* carry */
    chosen[j++] ← 0;
    num_chosen−−;
  }
  if (j ≡ num_choices) done ← 1;
  else {
    chosen[j] ← 1;
    num_chosen++;
  }
}
```

This code is used in chunk 84.

¶ If M_v is even, the middle row $v_m = M_v/2$ is its own image. We enumerate all nonempty subsets of it that are symmetric under $i \leftrightarrow W-1-i$: we choose among the positions in the right half, including the center. A chosen position stands for two cells, except the center. The first row to be built is $v_0 = v_m + 1$.

86 ⟨ Prepare the choices for the middle row 86 ⟩ ≡

```
int vm ← M_v/2;
for (int i ← 0; i < W; i++)
  if (((u_min + i − vm) % 2 + 2) % 2 ≡ 0 ∧ 2 ∗ (u_min + i) ≥ M_u) choice[num_choices++] ← i;
max_chosen ← (NMAX − 2 ∗ min_rows + 1)/2;
first_row ← vm + 1;
FIRST_ROW ← first_row;
```

This code is used in chunk 84.

¶ Every cell of the middle row is a component of its own; the partner of a cell is its mirror cell, and the center cell, if present, is the central component. The weight is the number of cells.

87 ⟨ Create the state for a subset of the middle row 87 ⟩ ≡

```
uint8_t label[MAXW], partner[MAXLAB];
int num_cells ← 0;
memset(label, 0, W);
for (int b ← 0; b < num_choices; b++)
  if (chosen[b]) label[choice[b]] ← label[W − 1 − choice[b]] ← 1;
for (int i ← 0; i < W; i++)
  if (label[i]) label[i] ← ++num_cells;
for (int i ← 0; i < W; i++)
  if (label[i]) partner[label[i]] ← label[W − 1 − i];      /∗ the cell itself at the center ∗/
if (num_cells + 2 ∗ min_rows ≤ NMAX) {
  canonical_state(label, partner, label[0] ≠ 0, &new_state);
  size_t target_state ← table_find_or_create(current, &new_state);
  current→first_record[target_state] ← new_record(current, num_cells, 1, 0);
}
```

This code is used in chunk 84.

¶ **The twin row.** If M_v is odd, the first row $v_0 = (M_v + 1)/2$ of P^+ is the twin row. Its image is the row $v_0 - 1$ directly below it. We choose an arbitrary nonempty subset of the twin row, and the row-by-row construction starts with the next row.

88 ⟨ Prepare the choices for the twin row 88 ⟩ ≡

```
first_row ← (M_v + 1)/2;
FIRST_ROW ← first_row;
for (int i ← 0; i < W; i++)
  if (((u_min + i − first_row) % 2 + 2) % 2 ≡ 0) choice[num_choices++] ← i;
max_chosen ← (NMAX − 2 ∗ (min_rows − 1))/2;
```

This code is used in chunk 84.

¶ The twin row and its image row together form a zigzag band with exactly one cell at each position: a cell of the twin row at the positions of one parity, and a cell of the image row at the positions of the other parity. The cell of the image row at position q is the image of the cell of the twin row at position $W-1-q$. We record in $band[q]$ whether the cell at position q of the band is occupied. Cells at adjacent positions of the band are neighbors. Hence the connected components of the known part are the maximal runs of occupied positions in the band. A run that consists of a single cell of the image row has no cell on the frontier, and it gets no label. The image of a run is the run at the mirror position: the r-th run from the left is the image of the r-th run from the right. Thus, a single cell of the twin row is a floating component, the run that contains the two middle positions of the band is the central component, and all other components are paired. There are no nested components.

The state is then treated like any state at the end of a row: we compute the lower bound and create the state if it is not too heavy. The weight is 2 for each cell.

89 ⟨ Create the state for a subset of the twin row 89 ⟩ ≡
```
uint8_t band[MAXW + 1];
int is_twin[MAXW], run_label[MAXW], num_runs ← 0, num_components ← 0;
memset(band, 0, W + 1);
memset(is_twin, 0, W * sizeof(int));
for (int b ← 0; b < num_choices; b++) {
  is_twin[choice[b]] ← 1;
  if (chosen[b]) band[choice[b]] ← band[W − 1 − choice[b]] ← 1;
}
for (int q ← 0; q < W; q++) {
  new_state.label[q] ← 0;
  if (¬band[q]) continue;
  if (q ≡ 0 ∨ ¬band[q − 1]) run_label[num_runs++] ← 0;      /* a new run */
  if (is_twin[q]) {
    if (¬run_label[num_runs − 1]) run_label[num_runs − 1] ← ++num_components;
    new_state.label[q] ← run_label[num_runs − 1];
  }
}
for (int r ← 0; r < num_runs; r++)
  if (run_label[r]) new_state.partner[run_label[r]] ← run_label[num_runs − 1 − r];
new_state.num_components ← num_components;
new_state.touches ← band[0];
int lowerbound ← bound(&new_state, 1, min_rows, W − 1, 1);
if (lowerbound ≥ 0 ∧ 2 * num_chosen + lowerbound ≤ NMAX) {
  size_t target_state ← table_find_or_create(current, &new_state);
  current→first_record[target_state] ← new_record(current, 2 * num_chosen, 1, 0);
}
```
This code is used in chunk 84.

7.4 Processing a row

The cells of row v are at the positions i for which $u_{\min} + i - v$ is even.

90 ⟨ Determine the cells of row k 90 ⟩ ≡
```
int v ← first_row + k − 1, num_row_cells ← 0;
for (int i ← 0; i < W; i++)
  if (((u_min + i − v) % 2 + 2) % 2 ≡ 0) row_cells[num_row_cells++] ← i;
```
This code is used in chunk 80.

91 ¶ ⟨ Decide cell *cell_number* of the row for all states 91 ⟩ ≡
```
int i ← row_cells[cell_number];
int last_in_row ← cell_number ≡ num_row_cells − 1;
table_initialize(next, i);
for (size_t source_state ← 0; source_state < current→num_states; source_state++) {
  ⟨ Extend state source_state by the two choices for cell i 92 ⟩
}
⟨ Swap the tables and update the statistics 95 ⟩
```
This code is used in chunk 80.

¶ A polyomino that is completed while row k is processed has its top in row $k - 1$.

92 ⟨ Extend state *source_state* by the two choices for cell i 92 ⟩ ≡
```
state_t old_state;
unpack(current→keys + source_state * BYTES_PER_KEY, current→decided_position, &old_state);
for (int occupied ← 0; occupied ≤ 1; occupied++) {
  int status ← step(&old_state, i, occupied, last_in_row, &new_state);
```

```
    if (status ≡ DEAD) continue;
    if (status ≡ COMPLETE) {
      ⟨ Record the polyominoes that are completed from state source_state 76 ⟩
      continue;
    }
    int added_weight ← occupied ? 2 : 0;
    ⟨ Add state source_state to the state new_state of the new table, with pruning 93 ⟩
  }
```

This code is used in chunk 91.

¶ The lower bound depends only on the new state and not on the state *source_state* from which it was reached. We therefore compute the bound only when the state is not yet in the new table, and we store it with the state. If already the smallest weight of the state *source_state* is too large, the new state is not created, and in this case the bound is not remembered. In a run with $N = 42$, a quarter of the 30.6 million new keys were already in the table. The bound was computed 22.9 million times, and 16.2 million states were created; the remaining 6.7 million keys were discarded, because they cannot be completed or because all weights were too large.

93 ⟨ Add state *source_state* to the state *new_state* of the new table, with pruning 93 ⟩ ≡

```
  uint8_t packed_key[MAX_BYTES_PER_KEY];
  pack_and_check(&new_state, next→decided_position, packed_key);
  size_t target_state ← table_find(next, packed_key);
  int lowerbound;
  if (target_state ≠ NO_STATE) lowerbound ← next→lowerbound[target_state];
  else {
    lowerbound ← bound(&new_state, k, min_rows, i, last_in_row);
    if (lowerbound < 0) continue;
    ⟨ Sharpen lowerbound by looking one cell ahead 94 ⟩
  }
  int limit ← NMAX − lowerbound − added_weight;
  if (current→records[current→first_record[source_state] − 1].weight > limit) continue;
  if (target_state ≡ NO_STATE) {
    target_state ← table_create(next, packed_key);
    next→lowerbound[target_state] ← lowerbound;
  }
  add_list(next, target_state, current, source_state, added_weight, limit);
```

This code is used in chunk 92.

7.5 Lookahead

The bound does not know which cells of the current row are already decided to be empty (Figure 6). This matters most at the kink: if the cell i is empty and there is a frontier cell right of it, that cell depends entirely on the next cell *next_i* of the row. We therefore try both possibilities for the next cell and take the smaller of the two bounds, including the weight of the cell. A completion in the lookahead counts only if it would be a valid polyomino.

94 ⟨ Sharpen *lowerbound* by looking one cell ahead 94 ⟩ ≡

```
  if (¬last_in_row ∧ ¬new_state.label[i] ∧ i + 1 < W ∧ new_state.label[i + 1]) {
      /* one step ahead: the next cell is present or not */
    int next_i ← row_cells[cell_number + 1], next_is_last ← cell_number + 1 ≡ num_row_cells − 1;
    int lookahead_bound ← 1 ≪ 20;
    state_t state_ahead;
    for (int next_occupied ← 0; next_occupied ≤ 1 ∧ lookahead_bound; next_occupied++) {
      int status_ahead ← step(&new_state, next_i, next_occupied, next_is_last, &state_ahead);
      if (status_ahead ≡ DEAD) continue;      /* a completion only counts if it is a valid polyomino */
      if (status_ahead ≡ COMPLETE ∧ ¬(new_state.touches ∧ vspan(k − 1) ≥ W)) continue;
      int bound_this_way ← 2 * next_occupied;
```

```
    if (status_ahead ≡ OK) {
      int lowerbound_ahead ← bound(&state_ahead, k, min_rows, next_i, next_is_last);
      if (lowerbound_ahead < 0) continue;
      bound_this_way += lowerbound_ahead;
    }
    lookahead_bound ← min(lookahead_bound, bound_this_way);
  }
  lowerbound ← max(lowerbound, lookahead_bound);
}
```

This code is used in chunk 93.

95 ¶ ⟨ Swap the tables and update the statistics 95 ⟩ ≡

```
table_t *swap ← current;
current ← next;
next ← swap;
MAX_STATES ← max(MAX_STATES, current→num_states);
size_t bytes ← table_bytes(current) + table_bytes(next);
if (bytes > MAX_BYTES)
  MAX_BYTES ← bytes, MAX_RECORDS ← current→num_records, MAX_KEY_BYTES ← BYTES_PER_KEY;
table_finish(current);      /* the new table is complete */
if (¬last_in_row ∧ (VERBOSITY ≥ 2 ∨ (VERBOSITY ≡ 1 ∧ wall_time( ) − LAST_REPORT_TIME ≥
    PROGRESS_SECONDS)))
  print_status_line("#cell", k, v, cell_number + 1, num_row_cells, current, next);
```

This code is used in chunk 91.

7.6 Reflection

If $M_u = 0$, the strip is symmetric under $u \to -u$, and at the end of a row the mirror image of a state is again a valid state with the same numbers of completions. We merge each state with its mirror image by keeping the smaller of the two in some fixed order, and adding the counts. This almost halves the number of states for these cases. (The option `-R` disables it.)

It is not sufficient to keep the smaller state of each pair and to double its counts. This would be correct at the first row where the merging takes place. In later rows, however, the tables contain only the successors of the states that were kept, and the count of a state t is no longer equal to the count of its mirror image t': some of the polyominoes that lead to t are represented by their mirror images, which lead to t'. Only the sum of the two counts is meaningful. We must therefore find the mirror state in the table, and we do this by building a new table.

We work with the coded states, in which the mirror image is easy to form: the codes are read from right to left, and `C_FIRST` is exchanged with `C_LAST`. (At the end of a row, all frontier cells are in the same row. Hence there are no adjacent frontier cells, and the code `C_CONTINUE` does not occur.) We compare the sequence of codes with the mirrored sequence lexicographically. It suffices to look at the left half: if the two sequences agree there, they agree everywhere.

96 ⟨ Merge every state with its mirror image 96 ⟩ ≡

```
if (MERGE_MIRROR_STATES ∧ M_u ≡ 0 ∧ current→num_states) {
  table_initialize(next, current→decided_position);
  for (size_t source_state ← 0; source_state < current→num_states; source_state++) {
    const uint8_t *packed_key ← current→keys + source_state * BYTES_PER_KEY;
    uint8_t mirror_key[MAX_BYTES_PER_KEY];
    uint8_t mirror_crossing[MAXLAB];      /* the crossing flags of the mirror image */
    coded_t coded;
    expand(packed_key, current→decided_position, &coded);
    int comparison ← 0;      /* positive if the mirror image is smaller than the state */
    for (int a ← 0, b ← W − 1; a < b ∧ comparison ≡ 0; a++, b−−)
      comparison ← (int) coded.code[a] − MIRRORED_CODE[coded.code[b]];
    if (comparison ≥ 0) {
```

```
      ⟨ Compute the flags of the mirror image and compare them with the flags of the state if
          comparison ≡ 0 98 ⟩
    }
    if (comparison > 0) {
      ⟨ Put the key of the mirror image into mirror_key 99 ⟩
      packed_key ← mirror_key;
    }
    size_t target_state ← table_find(next, packed_key);
    if (target_state ≡ NO_STATE) target_state ← table_create(next, packed_key);
    add_list(next, target_state, current, source_state, 0, NMAX);
  }
  table_t *swap ← current;
  current ← next;
  next ← swap;
  table_finish(current);
}
```

This code is used in chunk 80.

97 ¶ ⟨ Global variables 8 ⟩ +≡

```
static const uint8_t MIRRORED_CODE[6] ← {C_EMPTY, C_CONTINUE, C_SINGLETON, C_LAST, C_MIDDLE,
    C_FIRST};
```

¶ In the mirror image, the components are numbered from right to left, in the order in which their last runs appear in the original state. We need the labels of the original state to find the flags that tell which components are crossing.

If the codes are symmetric, the state and its mirror image have the same components, but they can still differ in these flags. (For example, of two symmetric single cells, one can be the central component, while the other is floating.) In this case we compare the two sequences of flags lexicographically.

98 ⟨ Compute the flags of the mirror image and compare them with the flags of the state if *comparison* ≡ 0 98 ⟩ ≡

```
state_t state;
decode(&coded, &state);
int mirror_label ← 0;
for (int b ← W − 1; b ≥ 0; b−−)
  if (coded.code[b] ≡ C_SINGLETON ∨ coded.code[b] ≡ C_LAST)
    mirror_crossing[++mirror_label] ← coded.crossing[state.label[b]];
for (int j ← 1; j ≤ coded.num_components ∧ comparison ≡ 0; j++)
  comparison ← (int) coded.crossing[j] − (int) mirror_crossing[j];
```

This code is used in chunk 96.

99 ¶ ⟨ Put the key of the mirror image into *mirror_key* 99 ⟩ ≡

```
coded_t mirror;
for (int a ← 0; a < W; a++) mirror.code[a] ← MIRRORED_CODE[coded.code[W − 1 − a]];
memcpy(mirror.crossing, mirror_crossing, MAXLAB);
mirror.num_components ← coded.num_components;
mirror.touches ← coded.touches;
compress(&mirror, current→decided_position, mirror_key);
```

This code is used in chunk 96.

8 The main program

```
-m M   count modulo M (default: modulo 2^64, or 2^128 with CNT128)
-o F   append the results to the file F (default results180.py)
-v V   progress report: 0 none, 1 per row (default), 2 per cell
-p S   with -v 1, report within a row every S seconds (default 60)
-R     do not merge mirror states
-C     check that packing and unpacking of every key are consistent
```

100 ⟨ Global variables 8 ⟩ +≡
```
static int MERGE_MIRROR_STATES ← 1;      /* merge mirror states? */
static const char *RESULT_FILE_NAME ← "results180.py";      /* -o */
static int CHECK_PACKING ← 0;      /* -C */
```

101 ¶ ⟨ The main program 101 ⟩ ≡
```
int main(int argc, char **argv)
{
  ⟨ Parse the command line 102 ⟩
  clock_t t0 ← clock( );
  START_TIME ← LAST_REPORT_TIME ← wall_time( );
  ⟨ Initialize the random bits for hashing 42 ⟩
  ⟨ Print the program parameters 104 ⟩
  if (argc − arg ≡ 4) {
    ⟨ Count a single case and print the result 105 ⟩
  }
  else {
    ⟨ Count all cases and add them up by type of center 106 ⟩
    ⟨ Append the results to the results file 107 ⟩
    ⟨ Print the table of results 108 ⟩
  }
  ⟨ Print the final statistics 109 ⟩
  return 0;
}
```
This code is used in chunk 2.

102 ¶* ⟨ Parse the command line 102 ⟩ ≡
```
int arg ← 1;
while (arg < argc ∧ argv[arg][0] ≡ '-') {
  if (¬strcmp(argv[arg], "-m") ∧ arg + 1 < argc) MODULUS ← strtoull(argv[++arg], 0, 10), arg++;
  else if (¬strcmp(argv[arg], "-o") ∧ arg + 1 < argc) RESULT_FILE_NAME ← argv[++arg], arg++;
  else if (¬strcmp(argv[arg], "-C")) CHECK_PACKING ← 1, arg++;
  else if (¬strcmp(argv[arg], "-v") ∧ arg + 1 < argc) VERBOSITY ← atoi(argv[++arg]), arg++;
  else if (¬strcmp(argv[arg], "-p") ∧ arg + 1 < argc) PROGRESS_SECONDS ← atof(argv[++arg]), arg++;
  else if (¬strcmp(argv[arg], "-R")) MERGE_MIRROR_STATES ← 0, arg++;
  else break;
}
if (arg < argc ∧ argv[arg][0] ≡ '-') {
  fprintf(stderr, "%s: unknown option %s\n", argv[0], argv[arg]);
  return 1;
}
if (arg ≥ argc) {
  fprintf(stderr, "usage: %s [options] N [Mx My umax]\n", argv[0]);
  return 1;
}
NMAX ← atoi(argv[arg]);
if (NMAX < 1 ∨ NMAX > MAXW) {
  fprintf(stderr, "%s: N must be between 1 and %d\n", argv[0], MAXW);
  return 1;
}
NMAX_plus_1 ← NMAX + 1;
⟨ Allocate the result vectors 75 ⟩
```
This code is used in chunk 101.

8.1 Limits

The program has several built-in limits. All of them are far beyond the range where the program can be useful.

- The strip has at most `MAXW` $= 160$ columns. Since strips of all widths up to N are needed, this means $N \le 160$, which is checked above.
- Component labels are stored in bytes. Hence a label cannot go beyond 255. A strip of width W has at most $\lceil W/2 \rceil + 1 \le 81$ components.
- The weight in a count record is a byte, which would allow $N \le 255$.
- The buffers for a packed key, and the tables of random numbers for hashing, are prepared for keys of at most `MAX_BYTES_PER_KEY` $= 41$ bytes. This is sufficient for all widths up to `MAXW`: The key for a strip of width W has 3 bits for each of the $\lfloor W/2 \rfloor + 1$ positions that can hold a frontier cell, one bit for each of the at most `MAX_COMPONENTS` $= \lceil W/2 \rceil + 1$ components, and the touched bit. For $W = 160$, these are $3 \cdot 81 + 81 + 1 = 325$ bits. (If `MAXW` is increased, `MAX_BYTES_PER_KEY` must be increased accordingly.)
- States and count records are indexed by 32-bit integers, so a table can hold fewer than $2^{32} \approx 4.3 \cdot 10^9$ records. (There are always at least as many records as states.) This is the limit that would be reached first, and the program stops when it happens. The run for $N = 71$ had $5.1 \cdot 10^7$ records at the peak. With a growth by a factor of 1.25 per unit of N, the limit corresponds to $N \approx 90$. Such a run would need about 70 times as much memory and time as the run for $N = 71$, that is, some 200 GB and 40 days.
- The hash values have 32 bits, which is sufficient for the tables that are possible under the previous limit.
- The possibilities for the first row are enumerated one by one. These are the symmetric subsets of the middle row, which are determined by about $W/4$ cells, or the subsets of the twin row, which has about $W/2$ cells; only subsets with at most *max_chosen* cells are generated. The largest numbers arise for $W \approx N/2$, where the size restriction is not effective, and up to about $2^{N/4}$ subsets are tried for the twin row. For $N = 71$, this is harmless (2^{18}), but it would become the bottleneck somewhere beyond $N = 120$.

8.2 Running the cases and writing the results

Both output streams start with the parameters of the run, beginning with the complete command line (including the name *argv*[0] of the program), so that every log file and every result file is self-describing.

104 ⟨ Print the program parameters 104 ⟩ ≡

```
for (int stream_number ← 0; stream_number < 2; stream_number++) {
  FILE *stream ← stream_number ? stdout : stderr;
  fprintf(stream, "# command:");
  for (int word ← 0; word < argc; word++) fprintf(stream, " %s", argv[word]);
  fprintf(stream, "\n# diag180 built %s %s; N=%d", __DATE__, __TIME__, NMAX);
  if (argc − arg ≡ 4)
    fprintf(stream, ", single case Mx=%s My=%s umax=%s", argv[arg + 1], argv[arg + 2], argv[arg + 3]);
  if (MODULUS) fprintf(stream, ", modulus %llu\n", (unsigned long long) MODULUS);
  else fprintf(stream, ", modulus 2**"COUNTER_BITS"\n");
  fprintf(stream, "# production version (cactus bound with height terms, looka\
      head at the kink); reflection merge %s; pack check %s\n",
      MERGE_MIRROR_STATES ? "on" : "off (-R)", CHECK_PACKING ? "on (-C)" : "off");
  fprintf(stream, "# output: results file %s, progress -v %d -p %g s\n", RESULT_FILE_NAME,
      VERBOSITY, PROGRESS_SECONDS);
  fprintf(stream, "# sizes: counter %zu bytes, count record %zu bytes\n", sizeof(counter_t),
      sizeof(record_t));
  fflush(stream);
}
```

This code is used in chunk 101.

¶ A single case can be run separately, for example for distributing the cases over the nodes of a cluster.

105 ⟨ Count a single case and print the result 105 ⟩ ≡

```
M_x ← atoi(argv[arg + 1]), M_y ← atoi(argv[arg + 2]), u_max ← atoi(argv[arg + 3]);
count_case( );
if (MODULUS)
  printf("# modulus %llu; n # (uspan<vspan) # (uspan=vspan)\n", (unsigned long long) MODULUS);
else printf("# modulus 2**"COUNTER_BITS"; n # (uspan<vspan) # (uspan=vspan)\n");
for (int n ← 1; n ≤ NMAX; n++)
  printf("%d %s %s\n", n, counter_to_string(PORTRAIT[n]), counter_to_string(SQUARE[n]));
```

This code is used in chunk 101.

¶ The cases are $(M_x, M_y) \in \{(0,0),(1,0),(1,1)\}$ and all $u_{\max}$ for which $W \le N$. The edge-centerd case $(1,0)$ is counted twice, since it stands also for its mirror image $(0,1)$.

106 ⟨ Count all cases and add them up by type of center 106 ⟩ ≡

```
  /* center types: 0 cell, 1 edge midpoint, 2 vertex */
counter_t *results ← calloc(3 * NMAX_plus_1, sizeof(counter_t));
for (M_x ← 0; M_x ≤ 1; M_x++)
  for (M_y ← 0; M_y ≤ M_x; M_y++) {
    /* the case (0,1) is the mirror image of (1,0): same counts, so we count (1,0) twice instead */
    int t ← M_x − M_y, center_type ← M_x + M_y, twice ← M_x ≠ M_y;
    for (u_max ← t; 2 * u_max − t + 1 ≤ NMAX; u_max++) {
      count_case( );
      for (int n ← 1; n ≤ NMAX; n++) {
        counter_t *entry ← &results[center_type * NMAX_plus_1 + n];
        *entry ← add_mod(*entry, add_mod(PORTRAIT[n], PORTRAIT[n]));
        *entry ← add_mod(*entry, SQUARE[n]);
        if (twice) {
          *entry ← add_mod(*entry, add_mod(PORTRAIT[n], PORTRAIT[n]));
          *entry ← add_mod(*entry, SQUARE[n]);
        }
      }
    }
  }
```

This code is used in chunk 101.

¶ The results are appended to a file in Python syntax, with lines of the form

```
Results[n, m, 'R180C'] = x
```

where m is the modulus. Runs with different moduli accumulate in the same file, and the final combination (Chinese remaindering, Burnside's lemma) can be done in Python with integers of unlimited size.

107 ⟨ Append the results to the results file 107 ⟩ ≡

```
  /* results file for the final combination (CRT, Burnside) in Python; appended, so runs with different moduli accumulate */
FILE *result_file ← fopen(RESULT_FILE_NAME, "a");
if (¬result_file) {
  perror(RESULT_FILE_NAME);
  return 1;
}
char modulus_string[32];
if (MODULUS) snprintf(modulus_string, sizeof modulus_string, "%llu", (unsigned long long) MODULUS);
else snprintf(modulus_string, sizeof modulus_string, "2**"COUNTER_BITS);
fprintf(result_file,
    "# diag180 N=%d modulus %s\ntry:\n    Results\nexcept NameError:\n    Results = {}\n",
    NMAX, modulus_string);
static const char *type_name[3] ← {"R180C", "R180M", "R180V"};
```

```
for (int n ← 1; n ≤ NMAX; n++)
  for (int center_type ← 0; center_type < 3; center_type++)
    fprintf(result_file, "Results[%d,␣%s,␣'%s']␣=␣%s\n", n, modulus_string, type_name[center_type],
        counter_to_string(results[center_type * NMAX_plus_1 + n]));
fclose(result_file);
printf("#␣modulus␣%s;␣results␣appended␣to␣%s\n", modulus_string, RESULT_FILE_NAME);
```

This code is used in chunk 101.

¶ The table of results has a line for each n with the total $\mathrm{fix}_\rho(n)$ and its three parts for the three types of centers.

108 ⟨ Print the table of results 108 ⟩ ≡

```
printf("#␣n␣fix(r180)␣R180C(center␣in␣a␣cell)␣R180M(edge␣midpoint)␣R180V(vertex)\n");
for (int n ← 1; n ≤ NMAX; n++) {
  counter_t total ← add_mod(add_mod(results[n], results[NMAX_plus_1 + n]),
      results[2 * NMAX_plus_1 + n]);
  printf("%d␣%s␣%s␣%s␣%s\n", n, counter_to_string(total), counter_to_string(results[n]),
      counter_to_string(results[NMAX_plus_1 +n]), counter_to_string(results[2*NMAX_plus_1 +n]));
}
```

This code is used in chunk 101.

109 ¶ ⟨ Print the final statistics 109 ⟩ ≡

```
fprintf(stderr, "#␣peak␣memory␣%.1f␣MB␣(both␣tables;␣%zu␣count␣records,␣key␣%zu␣bytes)\n",
    MAX_BYTES/1 · 10^6, MAX_RECORDS, MAX_KEY_BYTES);
fprintf(stderr, "#␣max␣states␣%zu,␣%.2fs\n", MAX_STATES, (double)(clock( ) − t0)/CLOCKS_PER_SEC);
```

This code is used in chunk 101.

9 Results and experiments

9.1 Correctness

The program has been checked in several ways. The results agree with a direct enumeration of all polyominoes by Redelmeier's algorithm [9] for $n \le 16$, and with an independent Python prototype of the same method for $n \le 36$. Runs with different moduli are consistent. For $n \le 59$, the totals of our run with $N = 60$ agree with the values in the OEIS (sequences A390653–A390655), which were computed by Shirakawa [10] with a completely different method; for $n \le 32$ we have also compared the three types of centers separately. In the full version of the program, different variants of the bound and of the lookahead can be selected, and they lead to the same counts; this is a strong test, since an invalid bound would lose polyominoes.

9.2 Tightness of the bound

For all states that arise for $N \le 22$, after every cell, we have computed the true minimum weight of a completion by exhaustive search. The bound, together with the lookahead at the kink, was equal to the true value for every one of the 87,842 states. Thus, at least in this range, no state is stored in vain.

9.3 Growth

Table 2 shows the maximum number of states in a table and the running time. The number of states grows by a factor of about 1.25 when N increases by one. The run for $N = 60$ took 66 minutes on a single core and needed 304 MB (before the mirror-image case $(0, 1)$ was eliminated, which saves one third of the time). The production run for $N = 71$ took 13.7 hours on a single core and needed 3 GB. It was done twice, modulo 2^{64} and modulo $2^{61} - 1$, with identical statistics. (The largest value has 68 bits.)

N	max. states	records	MB	time (s)
16	70	146	0.0	0.01
20	219	409	0.0	0.03
24	550	1,225	0.1	0.16
28	1,417	3,350	0.2	0.62
32	3,674	8,308	0.4	1.72
36	9,793	20,223	1.1	5.34
40	21,191	44,777	2.8	16.32
44	55,629	130,869	7.4	49.63
48	147,608	352,534	21.3	149.08
60	2,137,770	4,689,188	304.1	3991 (other machine, see text)
71	19,724,251	51,445,692	2959.3	49264 (a third machine, see text)

Table 2: Maximum number of states in one table, number of count records at the peak, memory for both tables, and running time on a single core (64-bit counters).

TODO: Run again after the program has settled.

9.4 Values

Table 3 lists the results for $n \le 71$. The values for $n \le 59$ agree with the previous values in the OEIS; the others are new. As a further check, we have derived the numbers of *free* polyominoes whose symmetry group is exactly the group generated by the 180° rotation (OEIS A006747 and its three parts A351615, A234008, A351616) from our numbers and from the known counts of the polyominoes with more symmetries. This involves divisions by 4 and by 2, which always came out exact, and the known terms for $n \le 59$ were reproduced.

9.5 Odd and even numbers of cells

The last column of Table 3 does not grow uniformly: the values come in pairs, with $\mathrm{fix}_\rho(2k+1)$ close to $\mathrm{fix}_\rho(2k)$. For even n, all three types of centers contribute, and type M alone accounts for about 80 % of the total; for odd n there is only type C with an occupied center cell. This is clear because the center cell is the only cell that is fixed by the rotation. The quotient $\mathrm{fix}_\rho(2k+1)/\mathrm{fix}_\rho(2k)$ is 1.019 for $2k = 10$ and decreases steadily: it is 1.001 for $2k = 28$, 0.9995 for $2k = 30$, and 0.983 for $2k = 70$. Thus, for all even $n = 2k$ with $30 \le n \le 70$, the values even go *down* from $n = 2k$ to $n = 2k+1$. Over two steps, the growth is very regular: $\mathrm{fix}_\rho(70)/\mathrm{fix}_\rho(68) \approx 4.002$, which is close to the growth $A(70)/A(69) \approx 4.005$ of the number of all fixed polyominoes in *one* step, in accordance with the square-root heuristic of the introduction (see also Table 1).

9.6 Saving space in larger runs

For $N = 71$, the memory requirement of 3 GB was no obstacle, and we have not tried to reduce it. (That run was made with an earlier version of the program, whose keys were twice as long.) At the peak, a state now needs about 62 bytes: 8 bytes for the packed key (62 bits for $W = 29$), 4 bytes for the list head, at least 8 bytes for its share of the hash array (two slots of 4 bytes), and 16 bytes for each of its count records (8 bytes for the counter, 4 for the link, 1 for the weight, 3 wasted by alignment), of which there are 2.6 per state. (This number is taken from the run for $N = 71$: at the moment of the largest memory usage, the larger table had $5.14 \cdot 10^7$ count records for $1.97 \cdot 10^7$ states.) (The actual memory at the peak of that run was 150 bytes per state of the larger table, because the old and the new table exist at the same time, and because the arrays grow by doubling and are not full. At that time, the program also kept the hash array of the old table, which is now released, see Section 5.) The number of states grows by a factor of about 1.25 per unit of N, that is, by a factor of 10 when N increases by 10. If space becomes scarce in larger runs, there are several options. We list them roughly in the order of increasing effort.

1. *Distributing the cases.* The cases $(M_x, M_y, u_{\max})$ are independent, and the program can already run a single case. This saves time when several machines are available, but not space: the peak is determined by the one or two worst strips.

2. *A fuller hash table.* The array *hash* has between two and four slots of 4 bytes per state, that is, 8 to 16 bytes. Linear probing still works well when the table is, say, 3/4 full (about 2.5 probes for a successful search and 8.5 for an unsuccessful one, instead of 1.5 and 2.5 at load 1/2). Allowing such a load factor

n	center in a cell (C)	edge midpoint (M)	vertex (V)	$\mathrm{fix}_\rho(n)$
1	1	0	0	1
2	0	2	0	2
3	2	0	0	2
4	0	6	1	7
5	7	0	0	7
6	0	20	4	24
7	24	0	0	24
8	1	70	14	85
9	86	0	0	86
10	6	254	50	310
11	316	0	0	316
12	27	944	186	1,157
13	1,178	0	0	1,178
14	114	3,556	704	4,374
15	4,442	0	0	4,442
16	472	13,520	2,685	16,677
17	16,896	0	0	16,896
18	1,926	51,792	10,312	64,030
19	64,716	0	0	64,716
20	7,789	199,566	39,841	247,196
21	249,283	0	0	249,283
22	31,348	772,640	154,642	958,630
23	964,708	0	0	964,708
24	125,802	3,003,322	602,468	3,731,592
25	3,748,031	0	0	3,748,031
26	503,948	11,713,496	2,354,592	14,572,036
27	14,610,484	0	0	14,610,484
28	2,016,677	45,816,460	9,227,412	57,060,549
29	57,118,440	0	0	57,118,440
30	8,065,830	179,656,144	36,246,300	223,968,274
31	223,859,532	0	0	223,859,532
32	32,251,819	706,012,804	142,671,941	880,936,564
33	879,285,686	0	0	879,285,686
34	128,955,260	2,779,850,708	562,600,898	3,471,406,866
35	3,460,424,846	0	0	3,460,424,846
36	515,657,653	10,964,180,124	2,222,092,230	13,701,930,007
37	13,642,112,667	0	0	13,642,112,667
38	2,062,335,114	43,311,037,828	8,789,217,470	54,162,590,412
39	53,865,266,960	0	0	53,865,266,960
40	8,250,061,654	171,325,633,988	34,809,890,792	214,385,586,434
41	212,982,833,863	0	0	212,982,833,863
42	33,011,955,188	678,563,531,242	138,027,690,188	849,603,176,618
43	843,202,561,450	0	0	843,202,561,450
44	132,132,934,138	2,690,629,691,636	547,891,969,992	3,370,654,595,766
45	3,342,114,787,290	0	0	3,342,114,787,290
46	529,032,164,150	10,679,964,797,364	2,176,951,996,156	13,385,948,957,670
47	13,260,804,930,438	0	0	13,260,804,930,438
48	2,118,783,780,213	42,432,707,438,606	8,657,510,269,463	53,209,001,488,282
49	52,667,141,592,300	0	0	52,667,141,592,300
50	8,488,385,208,118	168,738,323,390,010	34,458,522,549,812	211,685,231,147,940
51	209,361,911,445,176	0	0	209,361,911,445,176
52	34,017,019,523,282	671,551,934,303,384	137,256,376,227,738	842,825,330,054,404
53	832,940,890,785,598	0	0	832,940,890,785,598
54	136,363,468,404,368	2,674,683,627,090,628	547,112,224,619,120	3,358,159,320,114,116
55	3,316,370,384,325,374	0	0	3,316,370,384,325,374
56	546,798,865,890,672	10,660,289,373,493,158	2,182,260,930,530,668	13,389,349,169,914,498
57	13,213,587,492,371,313	0	0	13,213,587,492,371,313
58	2,193,217,495,584,004	42,515,621,761,219,428	8,709,713,766,597,954	53,418,553,023,401,386
59	52,682,486,874,469,758	0	0	52,682,486,874,469,758
60	8,799,475,720,692,402	169,665,022,179,518,808	34,781,678,682,411,048	213,246,176,582,622,258
61	210,174,787,044,287,374	0	0	210,174,787,044,287,374
62	35,314,186,539,986,938	677,459,634,769,025,376	138,973,126,432,505,228	851,746,947,741,517,542
63	838,970,377,476,893,422	0	0	838,970,377,476,893,422
64	141,760,480,848,748,369	2,706,490,511,006,608,570	555,559,273,399,502,783	3,403,810,265,254,859,722
65	3,350,801,724,691,573,155	0	0	3,350,801,724,691,573,155
66	569,207,945,412,562,900	10,818,015,990,722,597,900	2,221,959,821,577,283,358	13,609,183,757,712,444,158
67	13,389,758,925,336,116,348	0	0	13,389,758,925,336,116,348
68	2,286,087,340,672,559,182	43,260,748,052,225,576,624	8,890,707,275,472,466,316	54,437,542,668,370,602,122
69	53,531,052,184,564,034,510	0	0	53,531,052,184,564,034,510
70	9,183,688,164,160,710,980	173,074,897,071,650,017,240	35,589,330,954,315,943,862	217,847,916,190,126,672,082
71	214,109,513,347,417,644,862	0	0	214,109,513,347,417,644,862

Table 3: The number of fixed polyominoes with n cells that are invariant under a rotation by 180°, by the type of center. The entries for $n \geq 60$ are new.

would save a few bytes per state. In the same spirit, the arrays could grow by a smaller factor than 2, or they could be allocated right away with a size estimated from the previous row, so that less of the allocated space remains unused.

3. *Smaller counters.* With 32-bit counters and arithmetic modulo a prime below 2^{32}, a count record can be packed into 8 bytes (32 bits for the counter, 24 for the link, 8 for the weight) instead of 16. The price is a larger number of runs: three moduli are needed for the 68-bit values at $n = 70$, instead of two. These runs are independent and can be done in parallel. A cheaper intermediate step is to store the records of a state contiguously instead of as a linked list, which removes the link.

4. *Separating the structure from the counting.* Knuth's programs POLYNUM and POLYSLAVE [6] split the work into two phases. The first program runs the transfer-matrix method without any counters: it only finds out which states exist and which state leads to which, and it writes this as a sequence of elementary commands (essentially "add counter a to counter b" and "clear counter a") to a file. The second program executes these commands on an array of counters. It needs neither keys nor a hash table, it reads the commands sequentially, and it can be run several times with different moduli, or once with large counters. The first phase, in turn, needs no counters, only the keys, and, in our setting, for each state the range of weights that survive the pruning.

5. *External storage.* The hash table can be replaced by sorting: the successor states of a row are written to files in sorted runs, and the runs are merged, adding up the counters of equal keys. Only sequential access is needed, so that the tables can live on disk, and the main memory holds just the buffers. A simpler variant keeps the hash table for the new row in memory but reads the old row, which is only scanned once, from a file.

6. *Splitting the table by the untouched part of the frontier.* While the cells in the left part of a row are decided, the frontier cells far to the right of the kink are not touched: their component labels may change, but the pattern of occupied and empty positions stays the same, whatever is decided on the left. Two states with different patterns in the right part can therefore never lead to the same state during this time. Hence, the states can be distributed into groups according to this pattern, and the groups can be processed one after the other, or in parallel, with one small table per group. Not the whole table is needed in memory at once. When the kink has passed the middle of the row, the roles are exchanged: now the left part of the new row is fixed, and the states are regrouped according to its pattern. This is the basis of Jensen's parallel algorithm [5], and it is also used by Barequet and Ben-Shachar [1], who cut the row into more than two parts. The pruning is compatible with this, since the bound is computed for each state separately.

7. *Splitting a case.* A case can be divided into independent subproblems according to the set of occupied cells in the middle row (or in the twin row), and the results are added. Each subproblem has fewer states, at the expense of the total running time, because states that are reached in several subproblems are processed several times. We have not tried how much this would save.

10 References

[1] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. *Algorithmica*, 88:Article 51, 2026. Preliminary version in: Proc. Symposium on Algorithm Engineering and Experiments (ALENEX), SIAM, 2024, pp. 133–143. doi:10.1007/s00453-026-01392-6.

[2] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited: Corrigendum and addendum. Preprint, October 2026. arXiv:2610.10632.

[3] Gill Barequet, Micha Moffie, Ares Ribó, and Günter Rote. Counting polyominoes on twisted cylinders. *Integers*, 6:Article A22, 2006. doi:10.5281/zenodo.8275374.

[4] Iwan Jensen. Enumerations of lattice animals and trees. *Journal of Statistical Physics*, 102:865–881, 2001. arXiv:cond-mat/0007239, doi:10.1023/A:1004855020556.

[5] Iwan Jensen. Counting polyominoes: a parallel implementation for cluster computing. In *Computational Science — ICCS 2003*, volume 2659 of *Lecture Notes in Computer Science*, pages 203–212. Springer, 2003. doi:10.1007/3-540-44863-2_21.

[6] Donald E. Knuth. POLYNUM and POLYSLAVE. CWEB programs, 2001. URL: https://cs.stanford.edu/~knuth/programs.html.

[7] Donald E. Knuth and Silvio Levy. *The CWEB System of Structured Documentation*. Addison-Wesley, 1994. URL: https://tug.ctan.org/info/knuth/cwebman.pdf.

[8] OEIS Foundation. The On-Line Encyclopedia of Integer Sequences. Sequences A000105, A001168, A006747, and A390653–A390655. URL: https://oeis.org.

[9] D. Hugh Redelmeier. Counting polyominoes: yet another attack. *Discrete Mathematics*, 36:191–203, 1981. doi:10.1016/0012-365X(81)90237-5.

[10] Toshihiro Shirakawa. Enumeration of polyominoes up to size $N = 59$. Preprint, October 2025. arXiv:2510.22446.

[11] Guy L. Steele, Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proc. ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 453–472. ACM, 2014. doi:10.1145/2660193.2660195.

Changed Chunks

The following chunks were changed by the change file: 58, 64, 72, 73, 94, 100, 102, 104, 111, 117.

Index

__DATE__: 104*.
__int128: 33.
__TIME__: 104*.
A: 83.
a: 15 16 17 29 35 62 70 71 96 99.
add_list: 49 93 96.
add_mod: 35 49 76 79 81 106 108.
added_weight: 49 92 93.
arg: 101 102* 104* 105.
argc: 101 102* 104*.
argv: 101 102* 104* 105.
atof: 102*.
atoi: 102* 105.
B: 83.
b: 15 17 35 42 62 70 71 87 89 98.
band: 89.
best: 62.
bit_position: 26.
bound: 66 73* 89 93 94*.
bound_this_way: 94*.
buffer: 36.
byte: 26.
bytes: 26 31 32 95.
BYTES_PER_KEY: 22 23 31 39 41 43 45 82 92 95 96.
c: 15 62 72*.
C_CONTINUE: 22 23 24 28 29 96 97.
C_EMPTY: 22 24 27 28 29 31 97.
C_FIRST: 22 24 28 29 31 96 97.
C_LAST: 22 24 27 28 29 96 97 98.
C_MIDDLE: 22 24 28 29 97.
C_SINGLETON: 22 24 28 29 31 97 98.
cactus_bound: 64* 67 73*.
calloc: 43 44 75 106.
can_hold_frontier_cell: 30 31.
canonical_state: 18 21 87.
capacity: 39 43.
cell_number: 79 80 91 94* 95.
center_type: 106 107.
CHECK_PACKING: 46 100* 102* 104*.
checked_realloc: 38 39 43 44 48.
choice: 84 86 87 88 89.
chosen: 84 85 87 89.
clock: 79 101 109.
clock_gettime: 79.
CLOCK_MONOTONIC: 79.
CLOCKS_PER_SEC: 79 109.
CNT128: 33.
code: 27 28 29 31 96 98 99.
coded: 28 29 31 32 96 98 99.
coded_t: 27 28 29 31 32 96 99.
compare_states: 15 46.
comparison: 96 98.
COMPLETE: 19 20 21 92 94*.
component: 70.
compress: 31 32 99.
connection_bound: 73*.
cost: 67 70 71 72*.
count: 37 48 49 76.
count_case: 80 105 106.
COUNTER_BITS: 33 104* 105 107.
counter_t: 33 35 36 37 48 74 75 79 80 82 104* 106 108.
counter_to_string: 36 77 79 105 107 108.
crossing: 27 28 29 31 71 98 99.
current: 76 79 80 83 84 87 89 91 92 93 95 96 99.
current_buffer: 36.
CURRENT_POSITION: 58* 59 73*.
CURRENT_ROW: 58* 59 67 73*.
CURRENT_TOUCHES: 58* 64* 69.
cycle_cost: 55 63 70 71.
DEAD: 19 20 21 92 94*.
decided_position: 30 31 32 37 44 46 47 92 93 96 99.
decode: 29 32 98.
digit: 36.
distance: 73*.
done: 79 84 85.
encode: 28 29 32.
end: 36.
entry: 106.
exit: 38 43 44 46 48 82.
expand: 31 32 96.
extra: 65 72*.
extra_this_way: 72*.
extra_with_option: 65.
fclose: 107.
fflush: 77 79 104*.
finish_cost: 64* 65.
first_record: 37 39 43 45 49 76 87 89 93.
first_row: 83 86 88 90.
FIRST_ROW: 58* 73* 86 88.
first_run: 28.
fopen: 107.
fprintf: 38 43 44 46 48 77 79 82 102* 104* 107 109.
free: 39 43 44.
FREE_HEIGHT: 58* 60 69 72*.
FREE_SCORE: 58* 60 69 72*.
from: 16.
frontier_row: 59 69 70.
gap: 55 63 70 71.
gap_score: 62 70.
get_bits: 26 31.
h: 41 60 61.
hash: 37 39 43 44 45 116.
HASH_BITS: 40 41 42.
hash_mask: 37 39 43 44 45.
hash_of_key: 40 41 43 45.
hash_size: 39 43.
HASH_SLOT: 45.
height: 64* 65 77.
height_term: 65 72*.
i: 18 20 21 28 29 31 45 47 68 69 71 73* 86 87 88 90 91.

is_central: 16 20.
is_floating: 16 20.
is_paired: 16.
is_twin: 89.
j: 16 18 20 28 29 30 31 51 59 61 62 70 73* 85 98.
join: 17 21.
k: 9 73* 79 80.
key: 41.
keys: 37 39 43 45 92 96.
kink: 67 70 73*.
l: 41 42.
lab: 28.
label: 13 15 16 17 18 20 21 28 29 67 68 69 73* 87 89 94* 98.
last_in_row: 21 73* 91 92 93 94* 95.
LAST_REPORT_TIME: 78 79 95 101.
last_run: 28 68 70.
left_distance: 72*.
limit: 93.
lookahead_bound: 94*.
lowerbound: 37 39 43 44 45 73* 89 93 94*.
lowerbound_ahead: 94*.
m: 31.
M_u: 8 82 86 96.
M_v: 8 77 82 86 88.
M_x: 8 77 79 80 82 105 106.
M_y: 8 77 79 80 82 105 106.
main: 101.
max: 4 60 61 63 64* 65 72* 73* 94* 95.
MAX_BYTES: 74 95 109.
MAX_BYTES_PER_KEY: 40 42 47 93 96 103.
max_chosen: 84 85 86 88 103.
MAX_COMPONENTS: 13 14 82 103.
MAX_KEY_BYTES: 74 95 109.
MAX_RECORDS: 74 95 109.
MAX_STATES: 74 95 109.
MAXLAB: 13 16 18 21 27 28 29 87 96 99.
MAXW: 8 13 16 21 27 58* 68 70 71 80 82 84 87 89 102* 103.
max1: 63 71.
max1_out: 63.
max2: 63.
memcmp: 15 45.
memcpy: 21 45 99.
memset: 18 31 80 82 84 87 89.
MERGE_MIRROR_STATES: 96 100* 102* 104*.
MIDDLE_GAP: 58* 71 72*.
MIDDLE_PEAK: 58* 71 72*.
MIDDLE_SCORE: 58* 71 72*.
min: 4 48 60 61 62 63 64* 65 72* 73* 94*.
min_rows: 73* 82 84 86 87 88 89 93 94*.
mirror: 99.
mirror_crossing: 96 98 99.
mirror_key: 96 99.
mirror_label: 98.
MIRRORED_CODE: 96 97 99.
missing_rows: 64*.
MODULUS: 34 35 102* 104* 105 107.
modulus_string: 107.
n: 77 79 81 105 106 107 108.
neighbor: 21.
new_label: 18 21.
new_one: 49.
new_record: 48 49 87 89.
new_state: 83 87 89 92 93 94*.
next: 37 48 49 76 79 80 83 91 93 95 96.
next_i: 94*.
next_is_last: 94*.
next_occupied: 94*.
NMAX: 8 77 79 81 82 86 87 88 89 93 96 102* 104* 105 106 107 108.
NMAX_plus_1: 8 75 80 82 102* 106 107 108.
NO_STATE: 45 47 93 96.
now: 79.
num_bits: 26.
num_cells: 79 87.
num_choices: 84 85 86 87 88 89.
num_chosen: 84 85 89.
num_components: 13 15 18 21 27 28 29 31 46 51 89 98 99.
num_crossing: 71.
num_gaps: 55 63 71.
num_max: 63.
NUM_MIDDLE: 58* 69 71 72*.
num_open_pockets: 70.
NUM_OPTIONS: 58* 63 65 69.
num_pairs: 71.
num_pocket_gaps: 70.
num_records: 37 39 44 48 79 95.
num_row_cells: 80 90 91 94* 95.
num_runs: 68 70 72* 89.
num_states: 37 39 43 44 45 79 80 91 95 96.
num_top: 70 71.
occupied: 21 92.
offset: 26.
OK: 19 20 21 94*.
old_label: 18.
old_state: 76 92.
option: 65.
OPTION_COST: 56 58* 63 65.
OPTION_PEAK: 56 58* 63 65.
OPTION_SCORE: 56 58* 63 65.
others_remain: 20.
p: 38.
pack: 32 46.
pack_and_check: 46 47 93.
packed_key: 45 46 47 93 96.
partner: 13 15 16 17 18 20 21 28 29 51 71 87 89.
partner_a: 17.
partner_b: 17.
peak: 63 70 71.
peak_of_max: 63 71.
peak_out: 63.
perror: 107.
pocket_first_gap: 70.
pocket_gap: 70.
pocket_label: 70.

pocket_peak: 70.
pocket_score: 70.
PORTRAIT: 74 75 76 79 81 82 105 106.
pos: 31.
previous: 49.
print_row_results: 77 81 82.
print_status_line: 79 80 95.
printf: 105 107 108.
PROGRESS_SECONDS: 78 95 102* 104*.
put_bits: 26 31.
q: 48 89.
r: 76 89.
reach_height: 55 60 63 69 71.
reach_score: 55 60 63 69 71.
realloc: 38.
record_capacity: 37 39 48.
record_t: 37 39 48 104*.
records: 37 39 48 49 76 93.
relabel: 16 17.
result: 18 21.
result_file: 107.
RESULT_FILE_NAME: 100∗ 102* 104* 107.
results: 106 107 108.
retire: 20 21.
retired_label: 20.
right_distance: 72∗.
row_a: 62.
row_b: 62.
row_cells: 80 90 91 94*.
row_from_left: 62.
row_from_right: 62.
ROW_RESULTS: 74 75 76 77 79 80 81 82.
run_end: 68 70 72*.
run_label: 68 70 89.
run_start: 68 70 72*.
s: 28 29 32 35 38 46 47.
s_index: 49.
sc: 60.
score: 63 64∗ 65 70 71.
score_of_max: 63 71.
score_out: 63.
separately: 64∗.
side_and_top_cost: 61 62 69.
side_distance: 73∗.
SIDE_DISTANCE: 58∗ 64* 72*.
slot: 45.
snprintf: 107.
some_component_is_crossing: 51.
source: 49.
source_state: 49 76 91 92 93 96.
SQUARE: 74 75 76 79 81 82 105 106.
stack: 29.
stack_size: 29.
START_TIME: 78 79 101.
state: 21 43 51 67 71 73∗ 98.
state_ahead: 94∗.
state_capacity: 37 39 43 44 45.
state_slot: 43.
state_t: 13 15 16 18 21 28 29 32 46 47 67 73* 83 92 94* 98.
status: 21 92.
status_ahead: 94∗.
stderr: 38 43 44 46 48 77 79 82 102* 104* 109.
stdout: 104*.
step: 21 92 94*.
strcmp: 102*.
stream: 104∗.
stream_number: 104∗.
strtoull: 102*.
sum: 63.
swap: 95 96.
t: 39 44 45 47 48 63 71 106.
t_index: 49.
table_allocate: 39 83.
table_bytes: 39 79 95.
table_create: 43 45 47 93 96.
table_find: 45 47 93 96.
table_find_or_create: 47 87 89.
table_finish: 44 84 95 96.
table_free: 39 80.
table_initialize: 39 44 84 91 96.
table_t: 37 39 44 45 47 48 49 79 83 95 96.
tag: 79.
target: 49.
TARGET_ROW: 58∗ 61 64* 67 73*.
target_state: 49 87 89 93 96.
timespec: 79.
to: 16.
top_gap: 70 71.
top_label: 70 71.
top_peak: 70 71.
top_row: 77.
top_row_number: 77.
top_score: 70 71.
total: 108.
touches: 13 15 18 21 27 28 29 31 67 69 70 72* 73* 76 89 94* 99.
ts: 79.
tv_nsec: 79.
tv_sec: 79.
twice: 106.
TWIN: 8 9 77 80 82 84.
type_name: 77 107.
t0: 101 109.
uint32_t: 37 40 41 43 44 48 49 76.
uint64_t: 33 34 42.
uint8_t: 13 16 17 18 20 21 26 27 31 32 37 41 45 46 47 67 73* 84 87 89 93 96 97.
$u_{\max}$: 8 79 80 82 105 106.
$u_{\min}$: 8 77 82 86 88 90.
unpack: 32 46 92.
unpacked: 46.
uspan: 7 74.
v: 79 90.
value: 26.
VERBOSITY: 77 78 80 95 102* 104*.
vm: 86.
vspan: 7 9 74 76 77 81 82 94*.
W: 8.

wall_time: 79 95 101.
weight: 37 48 49 76 93.
weight_limit: 49.
word: 104*.
x: 36 42.
z: 42.

List of Refinements

⟨ Add state *source_state* to the state *new_state* of the new table, with pruning 93 ⟩ Used in chunk 92.
⟨ Add the central term and the terms for the side and the top 72 ⟩ Used in chunk 67.
⟨ Add the results of this row to the totals and report them 81 ⟩ Used in chunk 80.
⟨ Allocate the result vectors 75 ⟩ Used in chunk 102*.
⟨ Append the results to the results file 107 ⟩ Used in chunk 101.
⟨ Compute the flags of the mirror image and compare them with the flags of the state if *comparison* ≡ 0 98 ⟩ Used in chunk 96.
⟨ Count a single case and print the result 105 ⟩ Used in chunk 101.
⟨ Count all cases and add them up by type of center 106 ⟩ Used in chunk 101.
⟨ Create the initial states 84 ⟩ Used in chunk 80.
⟨ Create the state for a subset of the middle row 87 ⟩ Used in chunk 84.
⟨ Create the state for a subset of the twin row 89 ⟩ Used in chunk 84.
⟨ Decide cell *cell_number* of the row for all states 91 ⟩ Used in chunk 80.
⟨ Determine the cells of row k 90 ⟩ Used in chunk 80.
⟨ Double the capacity of table t and rehash all states 43 ⟩ Used in chunk 45.
⟨ Extend state *source_state* by the two choices for cell i 92 ⟩ Used in chunk 91.
⟨ Find the runs of the frontier; if the frontier is empty, return 0 68 ⟩ Used in chunk 67.
⟨ Generate the next subset for which the size *num_chosen* does not exceed *max_chosen*; set *done* if finished 85 ⟩ Used in chunk 84.
⟨ Global variables 8 14 23 34 40 58 74 78 97 100 ⟩ Used in chunk 2.
⟨ Include files 3 ⟩ Used in chunk 2.
⟨ Initialize the height computation with the frontier cells 69 ⟩ Used in chunk 67.
⟨ Initialize the random bits for hashing 42 ⟩ Used in chunk 101.
⟨ Local variables of *count_case* 83 ⟩ Used in chunk 80.
⟨ Merge every state with its mirror image 96 ⟩ Used in chunk 80.
⟨ Parse the command line 102 ⟩ Used in chunk 101.
⟨ Pay for the outer paths, the rings and the middle 71 ⟩ Used in chunk 67.
⟨ Prepare the choices for the middle row 86 ⟩ Used in chunk 84.
⟨ Prepare the choices for the twin row 88 ⟩ Used in chunk 84.
⟨ Print the final statistics 109 ⟩ Used in chunk 101.
⟨ Print the program parameters 104 ⟩ Used in chunk 101.
⟨ Print the table of results 108 ⟩ Used in chunk 101.
⟨ Put the key of the mirror image into *mirror_key* 99 ⟩ Used in chunk 96.
⟨ Record the polyominoes that are completed from state *source_state* 76 ⟩ Used in chunk 92.
⟨ Return −1 if all components of *state* are floating 51 ⟩ Used in chunk 73*.
⟨ Scan the runs from left to right and pay for the pockets 70 ⟩ Used in chunk 67.
⟨ Set up the geometry of the case 82 ⟩ Used in chunk 80.
⟨ Sharpen *lowerbound* by looking one cell ahead 94 ⟩ Used in chunk 93.
⟨ Subroutines 9 15 16 17 18 20 21 26 28 29 30 31 32 35 36 38 39 41 44 45 46 47 48 49 59 60 61 62 63 64 65 67 73 77 79 80 ⟩ Used in chunk 2.
⟨ Swap the tables and update the statistics 95 ⟩ Used in chunk 91.
⟨ The main program 101 ⟩ Used in chunk 2.
⟨ Type definitions 13 19 24 27 33 37 ⟩ Used in chunk 2.